



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

基于多元时序的分布式集群异常检测研究

作者姓名：杨婧雯

指导教师：裴昶华

学位类别：工程硕士

学科专业：计算机技术

培养单位：中国科学院大学计算机网络信息中心

2026 年 6 月

Research on Anomaly Detection for Distributed Clusters Based
on Multivariate Time Series

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of Engineering
in Computer Technology

By

YANG Jingwen

Supervisor: Prof. PEI Changhua

Computer Network Information Center, Chinese Academy of Sciences

June, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。承诺除文中已经注明引用的内容外，本论文不包含任何其他个人或集体享有著作权的研究成果，未在以往任何学位申请中全部或部分提交。对本论文所涉及的研究工作做出贡献的其他个人或集体，均已在文中以明确方式标明或致谢。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院大学有关收集、保存和使用学位论文的规定，即中国科学院大学有权按照学术研究公开原则和保护知识产权的原则，保留并向国家指定或中国科学院指定机构送交学位论文的电子版和印刷版文件，且电子版与印刷版内容应完全相同，允许该论文被检索、查阅和借阅，公布本学位论文的全部或部分内容，可以采用扫描、影印、缩印等复制手段以及其他法律许可的方式保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘 要

关键词：分布式集群，异常检测，多元时序，曲线相似性，自适应阈值

Abstract

Key Words: Distributed Cluster, Anomaly Detection, Multivariate Time Series, Curve Similarity, Adaptive Threshold

目 录

第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 论文研究内容和贡献	4
1.3.1 主要研究内容	4
1.3.2 主要贡献	4
1.4 论文组织结构	5
第 2 章 相关背景知识与理论基础	7
2.1 时间序列异常概述	7
2.1.1 时间序列异常数据简介	7
2.1.2 时间序列异常定义与分类	8
2.1.3 时间序列异常检测面临的挑战	8
2.2 时序异常检测算法	9
2.2.1 单变量时序异常检测	9
2.2.2 多元时序异常检测	11
2.3 分布式集群监控数据与异常检测	12
2.3.1 分布式集群概述	12
2.3.2 集群监控数据特点	13
2.3.3 分布式异常检测系统架构	13
2.3.4 异常检测方法在集群数据中的应用与挑战	14
2.4 本章小结	14
第 3 章 面向多节点单指标的异常检测算法	15
3.1 多节点单指标数据集构建	15
3.1.1 数据来源与采集	15
3.1.2 数据集预处理与标注	16
3.1.3 数据集统计与可视化	16
3.2 现有异常检测方法评估	17
3.2.1 异常检测基线算法	17
3.2.2 基于曲线相似性的异常检测方法	18
3.2.3 实验结果对比分析	20
3.3 参数敏感性分析与优化策略	21

3.3.1 阈值敏感性实验与分析	21
3.3.2 参数依赖性与泛化挑战	22
3.4 CSAD-AT: 基于曲线相似性的自适应阈值异常检测框架	23
3.4.1 经典 SPOT 算法原理与局限性分析	23
3.4.2 I-SPOT: 基于包容性极值建模的自适应阈值算法	24
3.4.3 多视角保守集成决策机制	25
3.5 实验验证与对比分析	26
3.5.1 实验设置	26
3.5.2 与基线算法实验结果对比	26
3.5.3 I-SPOT 算法的自适应阈值效果验证	27
3.5.4 CSAD-AT 框架的综合性能评估	27
3.6 本章小结	28
第 4 章 数值-形状双视角融合的多指标多节点异常检测	29
4.1 集群多指标多节点数据集及异常检测算法框架	29
4.1.1 多指标多节点数据集概述	29
4.1.2 问题分析与算法框架	31
4.2 基于数值特征的异常检测算法	32
4.2.1 基于欧氏距离的单指标异常度量	32
4.2.2 多指标加权融合策略	33
4.2.3 算法实现与效率优化	34
4.3 基于形状特征的异常检测算法	35
4.3.1 经典 SAX 算法原理	35
4.3.2 基于 SAX 算法的符号化表示与模式匹配	35
4.3.3 算法效率优化	36
4.4 多算法融合与故障定位	36
4.4.1 加权融合设计	36
4.4.2 故障定位与根因分析建议	37
4.5 实验验证与对比分析	37
4.5.1 实验设置	37
4.5.2 方法结果对比分析与案例研究	37
4.5.3 融合方法优势分析	38
4.5.4 算法效率实验对比	38
4.6 本章小结	38

第 5 章 基于曲线相似性的异常检测系统	41
5.1 系统设计	41
5.1.1 功能模块设计	41
5.1.2 系统架构设计	42
5.1.3 数据库设计	43
5.2 开发环境与技术选型	46
5.3 系统实现	48
5.3.1 用户与权限管理模块	48
5.3.2 数据接入与管理模块	48
5.3.3 预处理服务模块	49
5.3.4 检测引擎模块	50
5.3.5 常态化监控模块	50
5.3.6 告警管理模块	52
5.3.7 可视化与数据看板模块	53
5.4 系统应用效果展示	54
5.4.1 系统性能评估	55
5.4.2 常态化监控运行效果	55
5.4.3 告警系统运行效果	56
5.5 本章小结	56
第 6 章 总结与展望	59
6.1 本文工作总结	59
6.2 未来研究展望	59
参考文献	61
附录一 附录中的公式	63
附录二 附录中的图表	65
致谢	67
作者简历及攻读学位期间发表的学术论文与其他相关学术成果 ..	69

图目录

图 1-1 论文研究内容框架图	5
图 2-1 时间序列异常类型分类示意图	9
图 3-1 测试数据集异常标注曲线（红色阴影部分为异常标注）	17
图 3-2 CSAD-AT 框架整体架构	23
附图 2-1 这是一个样图	65

表目录

表 3-1 数据集统计数据	17
表 3-2 基线算法与本文方法在 RPC Router 连接数数据集上的异常检测效果对比	20
表 3-3 欧氏距离方法在不同阈值和时间窗口下的异常检测分数	21
表 3-4 AE 嵌入后异常检测分数	21
表 3-5 DBSCAN 异常检测分数	22
表 3-6 CSAD-AT 框架保守集成策略的检测效果	26
表 3-7 I-SPOT 自适应阈值与固定阈值方法的性能对比	27
表 4-1 故障类别及数量分布	30
表 4-2 监控指标列表及说明	30
表 4-3 多指标多节点异常检测方法对比	38
表 4-4 算法效率对比	38
表 5-1 用户表	43
表 5-2 集群配置表	44
表 5-3 检测任务表	44
表 5-4 检测结果表	44
表 5-5 告警规则表	45
表 5-6 告警记录表	45
表 5-7 系统代码目录结构	47
表 5-8 系统检测性能（单位：秒）	55
表 5-9 常态化巡检端到端耗时分布（30 节点 × 500 步）	55
附表 2-1 这是一个样表	65
附表 2-2 这是一个样表	65

符号列表

字符

Symbol	Description	Unit
R	the gas constant	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_v	specific heat capacity at constant volume	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_p	specific heat capacity at constant pressure	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
E	specific total energy	$\text{m}^2 \cdot \text{s}^{-2}$
e	specific internal energy	$\text{m}^2 \cdot \text{s}^{-2}$
h_T	specific total enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
h	specific enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
k	thermal conductivity	$\text{kg} \cdot \text{m} \cdot \text{s}^{-3} \cdot \text{K}^{-1}$
S_{ij}	deviatoric stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
τ_{ij}	viscous stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
δ_{ij}	Kronecker tensor	1
I_{ij}	identity tensor	1

算子

Symbol	Description
Δ	difference
∇	gradient operator
$\delta^\pm$	upwind-biased interpolation scheme

缩写

CFD	Computational Fluid Dynamics
CFL	Courant-Friedrichs-Lewy
EOS	Equation of State
JWL	Jones-Wilkins-Lee
WENO	Weighted Essentially Non-oscillatory
ZND	Zel'dovich-von Neumann-Doering

第 1 章 绪论

1.1 研究背景及意义

分布式集群（如 Hadoop、Kubernetes）是结合了分布式系统和集群系统优点的一种架构，在分布式集群中，不同的服务器负责不同的任务，同时每个任务可以由多个服务器组成的集群来处理。分布式集群是支撑现代云计算与大数据处理的核心基础设施，其稳定性直接关系到上层业务的连续性。此类集群在运行中会产生海量的多维度监控时序数据（如 CPU 负载、内存使用等），这些数据是感知系统健康状态、诊断异常的关键。

然而，随着集群规模不断扩大、架构动态性增强以及业务负载波动加剧，传统的基于阈值的异常检测方法因依赖先验知识、适应性差，导致误报和漏报率高，难以满足生产环境需求。尽管基于深度学习的方法能自动学习复杂模式，但在实际生产环境中，一个核心挑战在于对于如 RPC 连接数这类与用户任务相关的指标，其“正常”的绝对范围和模式难以统一定义，会随系统整体负载发生无规律波动，若使用基于历史数据训练的模型，会在整体高负载时产生大量误报，严重干扰运维效率。在分布式集群中，由于负载均衡机制的作用，在系统正常运行时，同一时刻各节点所承受的负载是相近的，各个节点指标曲线往往表现出极大的相似性，在运维实践中，运维人员常通过对比分布式集群中的同一指标曲线来判断异常：与其他变化模式相差很大的“离群曲线”更可能是异常。本研究正是基于这一洞察，旨在将“分布式集群相似性”这一先验知识系统性地引入多元时序异常检测模型，通过对比集群内多个节点在同一监控指标上的行为曲线，来识别其中显著偏离“群体一致性”模式的个别节点或子集，以提升异常检测的效果。

具体而言，本研究针对的是多节点时序数据集，即对同一指标（如 CPU 使用率）采集自集群中多个节点的、具有时间对齐关系的多条时序曲线。该数据集不仅包含每个节点自身的时序变化模式，更隐含了节点之间的运行关联与协同规律。如何有效捕捉并利用这种分布式关联，构建能够区分节点个体异常与集群整体波动的检测模型，是本研究要解决的关键问题。

本研究的意义在于，从方法和实践应用两个层面为分布式集群异常检测提供了新的思路与解决方案。在方法层面，本研究跳出了传统异常检测聚焦于“时间”与“指标”二维分析的框架，创新性地将“集群节点”这一空间维度系统性地引入模型，构建了“时间 $\times$ 指标 $\times$ 节点”的三维分析范式。这一范式将集群内在的、由负载均衡机制保证的节点行为相似性，从一种隐性的运维经验转化为一种显式的、可计算的先验知识，为处理动态负载场景下的异常检测问题提供了新的视角。在实践应用层面，所提出的方法直接针对生产环境中负载敏感型指标检测的痛点，通过识别“离群曲线”而非定义“绝对正常”，能够有效降低因集群整体负载波动引发的误报，提升检测结果的可靠性。基于真实运维数据构建数据集并

验证算法，既保证了方法在实际业务场景下的有效性，也为后续研究提供了数据基础。本研究结果不仅有助于实现更精准的故障定位与预警，缩短平均故障恢复时间，提升业务连续性，也为提升真实集群运维体系智能化提供了核心的检测能力支撑，具有重要的工程应用价值。

1.2 国内外研究现状

时间序列异常检测是数据挖掘和机器学习领域的重要研究方向，旨在识别时间序列数据中偏离正常行为模式的异常点或异常序列。随着物联网、工业 4.0 和金融科技的发展，时间序列数据规模呈指数级增长，对高效、准确的异常检测方法提出了更高要求。纵观该领域的发展历程，研究方法呈现出清晰的演进脉络：从基于严格统计假设的传统方法，到依赖特征工程的经典机器学习方法，再到能够端到端自动学习复杂模式的深度学习方法，体现了从模型驱动向数据驱动的智能学习范式转变。

传统统计方法奠定了异常检测的理论基石，其核心思想是基于概率分布假设进行统计推断，将显著偏离统计特性的观测点判定为异常。最具代表性的是 3-sigma 准则，该方法假设数据服从正态分布，根据切比雪夫不等式的推论，正常数据落在均值 μ 加减三倍标准差 σ 区间内的概率约为 99.7%，因此将落在该区间之外的点视为异常。然而，该方法对分布假设敏感，且难以处理存在多个离群值的情况。针对单个离群值的检测，Grubbs 检验^[1] 提供了更严谨的统计框架，其检验统计量定义为 $G = \frac{\max |x_i - \bar{x}|}{s}$ ，通过与基于样本量和显著性水平确定的临界值比较来判断异常，该方法适用于样本量较小且仅含单个离群值的场景。Dixon 检验^[2]（又称 Q 检验）则通过计算可疑值与相邻值的差异相对于数据全距的比例来判断异常，其计算简便且对极端值检测具有较好的鲁棒性。对于多变量场景，上述单变量方法存在明显局限，因为异常点可能在各单变量上并不极端，但其变量组合模式偏离正常关联结构。Mahalanobis 距离^[3] 通过引入协方差矩阵来度量样本点偏离分布中心的程度，其定义为 $D_M = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$ ，能够有效识别这类多变量关联异常。此外，基于时序模型的方法如 ARIMA^[4] 通过差分运算将非平稳序列转化为平稳序列，再结合自回归和移动平均成分建模时间序列的自相关结构，将预测残差作为异常判据；分解方法如 STL^[5] 采用局部加权回归将时间序列分解为趋势、季节和残差三个成分，通过分析残差成分中的极端值来识别异常。传统统计方法原理简单、可解释性强，但严重依赖分布假设，在面对复杂非线性模式和高维数据时能力有限。

经典机器学习方法拓展了异常检测的实用边界，其核心转变是从基于分布假设转向基于数据特征的相似性或密度度量。距离基方法直接利用距离度量分析样本间的相似性，其基本假设是异常点与正常点之间的距离较大。K 近邻方法^[6] 通过计算每个样本到其第 K 个最近邻居的距离作为异常分数，距离越大则越可能是异常。密度基方法则从局部密度的角度刻画异常，其中局部离群因子 (LOF)^[7] 是最具代表性的算法，它通过比较样本点的局部可达密度与其邻居的局

部可达密度来计算离群因子，LOF 值显著大于 1 表明该点相对于其邻域更加稀疏，因而更可能是异常。与距离基方法相比，LOF 能够更好地处理密度分布不均匀的数据。隔离基方法以孤立森林^[8]为代表，其核心思想源于一个直观的观察：异常点由于“少且不同”的特性，在随机分割构成的树结构中更容易被隔离出来。具体而言，算法通过随机选择特征和分割点递归划分数据空间构建隔离树，异常点由于偏离正常数据密集区域，通常只需较少的分割次数即可被隔离，因此其在树中的平均路径长度较短。孤立森林的时间复杂度为 $O(n \log n)$ ，相比需要计算成对距离的方法具有显著的效率优势，因此在工业界得到广泛应用。这些经典方法的共同优势在于无需大量标注数据，但效果依赖于精心设计的特征工程，且由于未显式建模时间依赖关系，难以有效捕捉时间序列中复杂的非线性关系和长期依赖。

深度学习方法实现了异常检测性能的显著突破^[9,10]，其核心创新是利用神经网络自动学习数据的内在表示，摆脱了对人工特征工程的依赖。重构型方法以自编码器及其变体为代表，其核心假设是：模型在正常数据上训练后学习到了正常模式的低维表示，当输入异常数据时由于其模式未被学习，解码器难以准确重构，从而产生较大的重构误差。OmniAnomaly^[11]在此基础上引入随机变分推断和归一化流，通过建模正常数据的复杂潜在分布来提升检测能力；USAD^[12]则采用两个共享编码器的解码器进行对抗训练，增强了模型对异常的敏感性。预测型方法利用序列模型的时序建模能力，通过预测未来时间步的取值并将预测误差作为异常分数。长短期记忆网络（LSTM）^[13]通过门控机制有效缓解了梯度消失问题，能够捕捉序列中的长期依赖关系。近年来，Transformer 架构凭借其强大的自注意力机制在时序建模中展现出优势，TranAD^[14]采用两阶段对抗训练机制，通过“自调节”策略放大异常样本的重建误差；Anomaly Transformer^[15]则提出关联差异性概念，利用异常点在时序关联模式上的特殊性进行检测。在多变量时序场景中，如何建模变量间的复杂依赖关系是核心挑战。图神经网络因能够显式建模变量间的拓扑关系而被广泛应用，GDN^[16]通过注意力机制自动学习传感器之间的图结构，并基于图卷积网络进行预测。此外，基于因果分析的方法将异常视为偏离正常因果机制的实例，CAROTS^[17]创新性地将因果关系融入对比学习框架，通过设计保持因果结构和破坏因果结构的两类数据增强策略，训练模型在潜在空间中区分正常与异常样本，同时支持异常根因定位。深度学习方法的主要优势在于强大的表征学习能力，但也面临训练不稳定、计算资源需求高、可解释性差等局限。

在研究趋势方面，当前领域呈现出多个重要发展方向。频域分析与时域分析的结合成为提升检测能力的新途径^[18]，通过傅里叶变换等方法从频率视角捕捉异常模式。对比学习与自监督学习的兴起为解决标签稀缺问题提供了新思路^[19]，通过设计有效的数据增强策略从无标签数据中学习鲁棒表征。可解释性与评估标准化受到越来越多的重视，因果推断方法被引入以增强模型透明度，专门化评估指标如 VUS-PR^[20]和综合性基准框架如 TAB^[21]不断涌现，为公平比较不同

方法提供了平台。随着数据规模的增长，分布式系统架构的发展也成为必然趋势，基于 Spark^[22]、Flink^[23] 等框架的系统实现了大规模时序数据的高效处理。

综上所述，现有研究虽然取得了显著进展，但绝大多数工作仍聚焦于时间与指标这两个维度构成的二维数据分析框架内。一个普遍的局限性在于，未能系统性地挖掘和利用分布式集群生产环境中广泛存在的多节点这一宝贵维度信息，忽视了同一集群内各节点指标行为模式的内在相似性及其对异常判别的辅助价值。因此，本研究旨在填补这一空白，探索一种基于时间、指标与集群节点的三维数据分析新范式，以期提升异常检测在真实动态运维场景中的鲁棒性。

1.3 论文研究内容和贡献

1.3.1 主要研究内容

本文围绕分布式集群多节点场景下的时序异常检测问题，开展了以下三个方面的研究工作。

第一，面向单指标多节点场景的异常检测算法研究。本文以 YARN 集群的 RPC Router 连接数指标为研究对象，构建了基于联通大数据生产环境的单指标多节点时序数据集。在此基础上，分别采用欧氏距离、自编码器嵌入距离和 DBSCAN 聚类三种方法度量节点曲线间的相似性，验证了基于群体一致性思想的异常检测方法的有效性。针对单一方法阈值敏感的问题，设计了基于逻辑与策略的集成学习框架，并引入改进的 SPOT 算法实现阈值的动态自适应调整。

第二，面向多指标多节点场景的异常检测算法研究。本文使用真实 GPU 集群故障数据集，设计了数值与形状双视角融合的异常检测框架。在数值视角方面，采用基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面，提出了改进的 SAX 符号化表示算法，通过取消分段聚合保留原始分辨率并采用严格距离计算规则来提升对形状变化的敏感性。最终通过加权融合两种视角的检测结果，实现对不同类型异常的全面覆盖。

第三，基于曲线相似性的异常检测系统设计与实现。本文设计了面向分布式集群的异常检测系统架构，实现了从数据采集、预处理、异常检测到结果展示的完整流程，并通过算法效率优化保障了系统在大规模集群环境下的实时性要求。

1.3.2 主要贡献

本文的主要贡献包括以下三个方面。

第一，本文提出了一种基于多节点曲线相似性的分布式集群异常检测方法。该方法突破了传统异常检测聚焦于时间与指标二维分析的框架，创新性地从集群节点这一空间维度系统性地引入模型，构建了时间、指标与节点的三维分析范式。通过利用分布式集群负载均衡机制所保证的节点行为相似性，将隐性的运维经验转化为显式的可计算先验知识。

第二，本文设计了数值与形状双视角融合的多指标异常检测框架。该框架从

数值幅度和曲线形态两个互补角度刻画节点异常行为，其中改进的 SAX 算法通过保留原始分辨率和采用严格距离计算规则增强了对形状变化的敏感性。通过加权融合两种视角的检测结果，该框架能够有效识别不同类型的节点异常。

第三，本文基于联通大数据真实生产环境构建了单指标多节点时序数据集，并使用阶跃星辰 GPU 集群故障数据集进行多指标多节点场景的验证。在这些数据集上的实验结果表明，本文所提方法在检测准确率方面显著优于现有的多元时序异常检测算法，为分布式集群异常检测提供了新的解决方案。

本文的研究内容框架如图1-1所示。

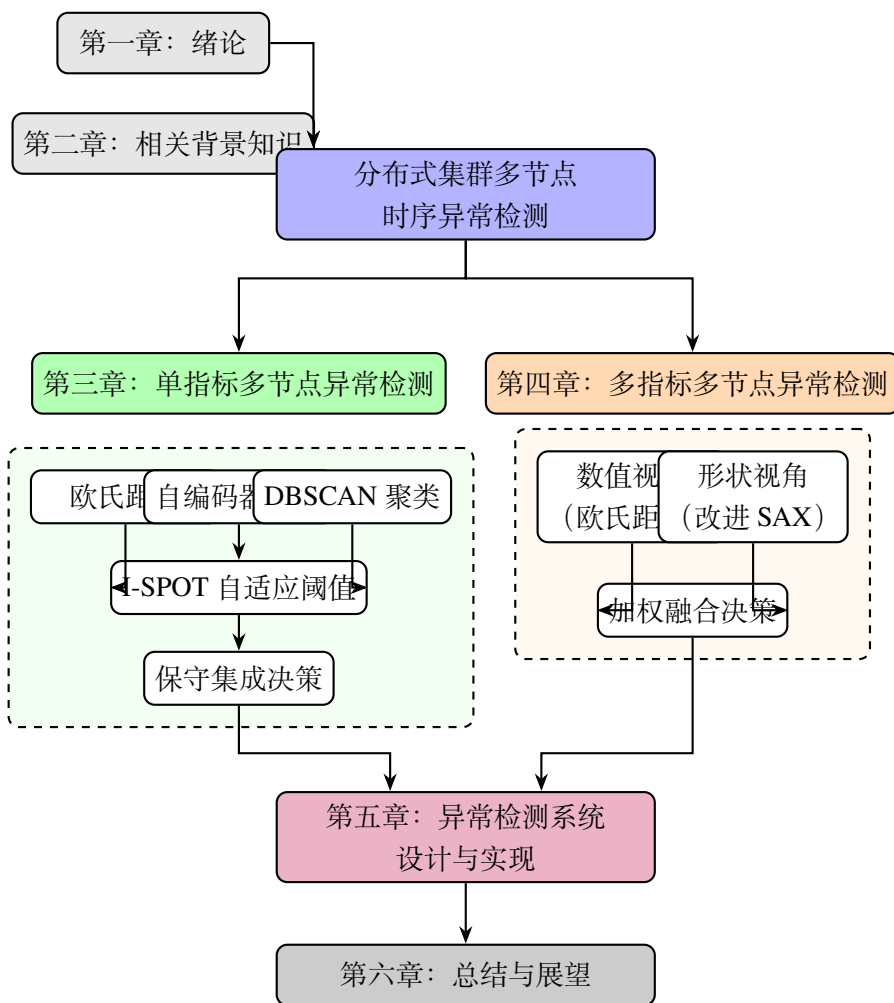


图 1-1 论文研究内容框架图

1.4 论文组织结构

本文共分为六章，各章内容安排如下。

第一章绪论部分阐述了本研究的背景与意义，回顾了多元时间序列异常检测领域的国内外研究现状，概述了本文的主要研究内容与贡献。

第二章介绍了相关背景知识与理论基础，包括时间序列异常的定义与分类、

主流的时序异常检测算法，以及分布式集群监控数据的特点与挑战。各章实验所采用的评估指标在对应的实验设置部分进行介绍。

第三章针对单指标多节点场景，详细介绍了数据集的构建过程、基于曲线相似性的异常检测方法，以及集成学习和自适应阈值等改进策略。

第四章针对多指标多节点场景，首先介绍了所使用的阶跃星辰 GPU 集群故障数据集，然后提出了数值与形状双视角融合的异常检测框架，详细阐述了基于欧氏距离的数值异常检测算法和基于改进 SAX 的形状异常检测算法，并介绍了多算法融合与故障定位方法。

第五章介绍了基于曲线相似性的异常检测系统，包括系统的总体流程设计、架构与关键技术、实现细节及应用效果展示。

第六章对全文工作进行总结，并对未来研究方向进行展望。

第2章 相关背景知识与理论基础

本章系统介绍时间序列异常检测的相关背景知识与理论基础。首先阐述时间序列异常的基本概念、数据特征和分类体系；其次回顾从传统统计方法到深度学习方法的时序异常检测算法演进；然后介绍分布式集群的架构特点、监控数据特性及异常检测面临的挑战；最后介绍时间序列异常检测的评估指标与基准测试框架。本章内容为后续章节的算法设计奠定理论基础。

2.1 时间序列异常概述

时间序列数据作为记录系统、过程或现象随时间演变信息的重要载体，广泛存在于工业监控、金融交易、网络安全及物联网等众多领域。对时间序列中的异常进行准确、及时的检测，是感知系统状态、预测潜在故障、保障业务稳定的关键技术。本节将系统阐述时间序列异常检测的定义、分类等核心概念。

2.1.1 时间序列异常数据简介

时间序列数据是按照时间顺序排列的一系列观测值的集合，其数学形式可表示为 $X = \{x_1, x_2, \dots, x_T\}$ ，其中 x_t 表示在时刻 t 的观测值， T 为序列长度。在单变量时间序列中，每个时刻的观测值为标量；在多元时间序列中，每个时刻的观测值为向量 $\mathbf{x}_t \in \mathbb{R}^D$ ，其中 D 表示变量维度（对于单变量序列， $D = 1$ ）。时间序列异常检测的目标是学习一个异常评分函数 $f(\cdot)$ ，并基于分数 $f(x_t)$ 或 $f(X_{(i,j)})$ （对于子序列）是否超过阈值 δ 来判断异常。

时间序列数据具有若干典型特征。首先是**时序相关性**，即相邻时刻的观测值之间往往存在较强的相关性，这种相关性可以是短期的自相关，也可以是长期的依赖关系。其次是**周期性和季节性**，许多时间序列数据呈现出周期性波动的规律，例如网络流量的日周期和周周期变化。第三是**趋势性**，时间序列可能呈现长期的上升或下降趋势。第四是**非平稳性**，时间序列的统计特性可能随时间发生变化，表现为均值、方差或相关结构的漂移。

多变量时间序列具有多维观察、时间依赖性、交叉依赖性、非平稳性以及可能包含不规则取样和缺失数据等关键特性。其异常检测需要同时分析时间维度和变量维度，建模变量间复杂的时空依赖关系，这比单变量场景更具挑战性。

在分布式集群监控场景中，时间序列数据通常以固定采样间隔连续采集，涵盖 CPU 使用率、内存占用、网络流量、磁盘读写等多个维度。这些监控指标共同反映了系统的运行状态，为故障诊断和性能优化提供了重要的数据基础。

2.1.2 时间序列异常定义与分类

时间序列中的异常是指与数据的正常行为模式存在显著偏离的观测点或观测片段，通常定义为明显偏离大多数样本的数据点。根据异常的时空范围和表现形式，可将时间序列异常划分为以下几类。

第一类是点异常，指时间序列中单个数据点的突变，表现为某个数据点显著偏离全局或局部正常数据点的分布。点异常可进一步分为全局点异常和局部点异常。全局点异常不依赖于局部上下文，显著偏离整体数据分布，例如数值突然飙升至历史最高点。局部点异常则在相邻时间点的局部上下文中偏离其周围数据模式，例如在夜间低峰期出现的中等流量峰值。

第二类是片段异常（模式异常），指时间序列中一段连续数据点的行为显著偏离正常模式，通常表现为波形、季节性或趋势的突变。这类异常包括波形异常、季节性异常和趋势异常，检测时需要关注子序列的整体模式而非单个点。例如，原本平稳的指标突然出现剧烈震荡，或者周期性指标的周期规律被打破。

第三类是变量间关联异常，这是多变量时间序列特有的异常类型，指变量间的关联关系随时间发生变化而导致的异常。单个变量可能不表现为点异常或片段异常，但与其他变量的关联模式（如相关性、因果关系）发生了异常变化。检测此类异常需要同时分析时间依赖性和变量间的相互依赖关系。

在分布式集群场景中，本文重点关注的是**节点级异常**，即在多节点环境下，个别节点的监控曲线偏离集群整体行为模式的情形。这类异常的识别需要同时考虑时间维度的变化规律和节点维度的群体一致性。

上述四种异常类型的示意图如图2-1所示。

2.1.3 时间序列异常检测面临的挑战

时间序列异常检测因数据特性、算法需求和应用场景的复杂性而面临诸多挑战。

在**数据层面**，主要挑战包括：（1）概念漂移，即数据分布会随时间动态变化；（2）数据维度高，特别是在多变量场景下存在“维度诅咒”问题；（3）噪声干扰，真实数据中往往包含各种噪声；（4）大规模数据处理，数据规模的爆炸式增长对算法的可扩展性提出了更高要求；（5）标签稀缺，异常数据本身稀少且标注成本高；（6）异常类型多样，不同类型的异常需要不同的检测策略。

在**算法层面**，则面临：（1）阈值选择困难，不同场景下最优阈值差异较大；（2）缺少根因定位数据集，现有公开数据集通常只标注了异常发生的时间段，但缺乏对异常具体成因的标注；（3）可解释性差，复杂的深度学习模型虽然检测性能优异，但其决策逻辑难以解释。

在**应用层面**，实时性/在线性受限、泛化性较差以及异常提前预警缺乏是主要难题。

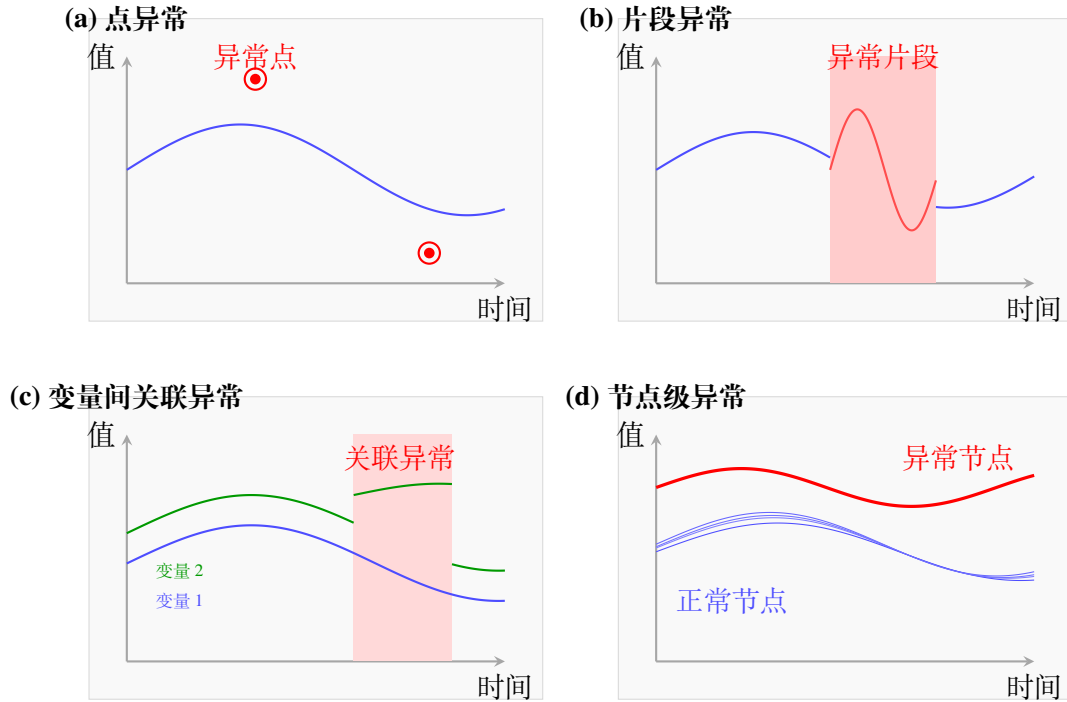


图 2-1 时间序列异常类型分类示意图

2.2 时序异常检测算法

时间序列异常检测方法可以从多个维度进行分类，主要包括学习范式、技术原理和数据维度。从学习范式角度，监督模式的选择取决于是否有标注数据：无监督方法（如孤立森林^[8]）无需任何标注数据，适用于新系统上线初期；半监督方法（如OCSVM、自编码器）仅需正常样本标注，在工业场景中最为常用；有监督方法则需要正常与异常样本均有标注，适用于已知特定异常模式的场景。

2.2.1 单变量时序异常检测

单变量时序异常检测旨在识别单条时间序列中的异常点或异常片段，其方法体系经历了从统计方法到机器学习方法再到深度学习方法的演进。

2.2.1.1 传统统计方法

传统统计方法建立在概率分布假设和时序模型之上，在数据量适中、模式相对稳定时具有良好的理论基础和可解释性。

基于分布假设的检验方法主要包括集中不等式和统计置信度检验。3-sigma准则假设数据服从正态分布，将落在 $(\mu - 3\sigma, \mu + 3\sigma)$ 区间之外的点视为异常。Grubbs 检验和 ESD (Extreme Studentized Deviate test) 是专门为单变量数据设计的异常值统计检验方法。

基于时序模型的预测方法以 ARIMA (自回归综合移动平均) 模型^[4] 为代表。该方法通过差分将非平稳序列转化为平稳序列，再结合自回归 (AR) 和移动平均 (MA) 成分进行建模。异常检测通过比较模型预测值与实际观测值的偏差来

实现，偏差超过设定阈值的点被判定为异常。

分解方法如 STL (Seasonal-Trend decomposition using Loess)^[5] 将时间序列分解为季节性、趋势性和残差三个部分。异常通常表现为残差分量中的极端值。这种方法直观，能有效处理周期性数据，但对季节性参数敏感，且传统实现实时性较差。

传统统计方法的优点是原理简单、计算效率高、可解释性强，但其假设时序数据服从特定的统计分布，在面对复杂非线性模式时效果有限。

2.2.1.2 经典机器学习方法

经典机器学习方法通过设计或选择特征度量，以无监督方式识别偏离正常模式的样本。

基于距离的方法如 KNN (K 近邻)，通过计算数据点与其 K 个最近邻居的距离来识别异常，距离过大的点被认为是异常的。

基于密度的方法以 LOF (局部离群因子)^[7] 为代表。LOF 计算每个点的局部离群因子，衡量其相对于邻居的稀疏程度，值远大于 1 通常表示异常。该方法善于处理局部密度变化大的场景。

基于树的方法中，孤立森林 (Isolation Forest)^[8] 因其高效性在工业界应用广泛。其核心思想是异常点“少且不同”，更容易在随机分割构成的树结构中被快速“孤立”出来（路径更短）。算法通过构建多棵隔离树，并基于样本的平均路径长度计算异常分数。

基于聚类的方法（如 K-means、高斯混合模型 GMM、DBSCAN^[24]）将数据点分组，远离聚类中心或属于极小簇的点被视为异常。这类方法的有效性高度依赖于聚类参数和质量。

这些经典方法的共同优势在于无需大量标注数据（无监督），计算效率相对较高。然而，它们通常难以捕捉时间序列中复杂的非线性关系和长期依赖，对参数选择敏感。

2.2.1.3 深度学习方法

深度学习方法利用神经网络自动学习数据的内在表示和复杂时序模式，通过重构或预测误差来识别异常^[9,10]。

LSTM (长短期记忆网络)^[13] 通过门控机制（遗忘门、输入门、输出门）有效缓解了传统 RNN 的梯度消失问题，能够捕捉序列中的长期依赖关系。在异常检测中，通常使用历史正常数据训练 LSTM 进行多步预测，将显著的预测误差作为异常判据。

自编码器 (AutoEncoder) 类方法通过编码器将输入压缩为低维潜在表示，再由解码器进行重构。模型在正常数据上训练，旨在最小化重构误差。当异常数据输入时，由于其模式未被学习，会产生较大的重构误差，从而被识别。变分自编码器 (VAE)^[25] 进一步引入了对数据潜在分布的概率建模。

TFMAE (时频掩码自编码器)^[26] 是应对“异常偏置”（训练数据含异常导致模型学习偏差）和“时序分布漂移”挑战的创新方法。其核心是采用基于窗口的时间掩码和基于幅值的频率掩码两种策略，有选择地预消除输入中的潜在异常。模型训练目标不是最小化传统重构误差，而是最小化时间掩码表示与频率掩码表示之间的对比差异，这种基于表示相似性的机制对分布偏移更具鲁棒性。

深度学习方法的主要优势在于其强大的表征学习能力，能自动捕获非线性与复杂依赖。其局限性包括训练不稳定、对数据量和计算资源需求高、超参数调优复杂以及模型决策可解释性差。

2.2.2 多元时序异常检测

多元时序异常检测 (Multivariate Time Series Anomaly Detection, MTSAD) 旨在识别多个相关变量构成的时序数据中的异常模式。相较于单变量时间序列，多变量数据不仅包含每个变量自身随时间变化的模式，还蕴含了复杂的变量间关联关系，这既是其检测能力提升的潜力所在，也构成了核心挑战。

2.2.2.1 基于重构的方法

OmniAnomaly^[11] 采用随机变分自编码器和平面归一化流来学习多元时间序列的正常模式分布，通过重构概率识别异常。该方法能够捕捉时间依赖性并建模正常数据的复杂分布。

USAD^[12] 基于对抗性训练的自编码器架构，通过引入对抗性训练来增强自编码器的鲁棒性，从而提升重构误差对异常的敏感性。

2.2.2.2 基于预测的方法

TranAD^[14] 是一种基于深度 Transformer 的异常检测与诊断模型。它采用两阶段对抗训练机制：第一阶段生成输入窗口的近似重构，其重建损失作为第二阶段的“关注分数”；第二阶段利用此分数重新运行推理，旨在放大重建误差。这种“自调节”机制有助于捕获鲁棒的多模态特征，并通过对抗训练提高稳定性。

Anomaly Transformer^[15] 通过关联差异性机制捕捉异常的时序特征，利用 Transformer 架构的强大自注意力机制捕捉时间序列长期依赖关系。

MTAD-GAT^[27] 结合了图注意力网络^[28] 和时间卷积网络，同时建模变量关联和时间依赖。

2.2.2.3 基于图神经网络的方法

图神经网络 (GNN)^[29,30] 天然适合对多变量时间序列中变量间的复杂关系进行显式建模，通过将变量视为图中的节点，关系视为边，GNN 可以学习节点和边的表示，以识别异常节点或子图。

GDN^[16] 通过学习传感器之间的图结构来检测异常。该方法通过图神经网络传播信息并预测未来值，能够捕捉变量之间的复杂依赖关系。

GST-Pro^[31] 针对现实中常存在缺失值的非规则采样多变量时间序列数据, 提出了基于神经控制微分方程的图时空过程建模方法。该框架能够从空间(变量间)和时间两个视角有效建模含缺失值的数据, 并采用一种新颖的基于分布的异常评分机制。

MGCL^[32] 专注于捕捉变量间的高阶交互关系, 它结合了多尺度时序建模和自适应超图结构, 以有效学习复杂的变量间和时序依赖关系。

2.2.2.4 基于因果分析的方法

从因果视角审视异常检测问题, 将异常视为偏离了数据生成常规因果机制的实例, 为提升检测的鲁棒性和可解释性提供了新思路。这类方法的核心创新在于将因果结构学习融入检测框架, 利用因果系统的模块化特性, 将复杂的多变量异常检测问题分解为一系列更简单、低维的子问题, 从而直接定位异常根源。

CAROTS^[17] 创新性地将因果关系概念融入对比学习框架。其核心是设计了两种数据增强器: 一种保留原始数据中的因果结构以生成代表正常波动的正样本; 另一种则有意扰动因果关系以生成模拟异常行为的负样本。通过这种基于因果关系的对比学习, 训练编码器在潜在空间中有效区分正常与异常样本。

CGAD^[33] 利用转移熵构建变量间的因果图结构, 并结合加权图卷积网络和因果卷积来同时捕捉因果与时间模式。

2.2.2.5 基于生成对抗网络的方法

MAD-GAN^[34] 基于 LSTM 的生成对抗网络^[35], 通过判别器得分和重构误差共同识别异常。该方法利用生成器和判别器的对抗训练来学习正常数据分布。

上述方法虽然在多元时序异常检测方面取得了显著进展, 但主要关注时间维度和指标维度, 未能充分利用分布式集群中多节点之间的行为相似性。本文的研究正是旨在弥补这一不足, 将节点维度纳入异常检测的分析框架。

2.3 分布式集群监控数据与异常检测

2.3.1 分布式集群概述

分布式集群是结合了分布式系统和集群系统优点的一种计算架构。在分布式集群中, 不同的服务器节点负责不同的任务, 同时每个任务可以由多个服务器组成的集群来协同处理。典型的分布式集群架构包括 Hadoop 生态系统中的 YARN 集群和容器编排领域的 Kubernetes 集群等。

以 YARN 集群为例, 其架构由资源管理器和多个节点管理器组成。资源管理器负责整个集群的资源调度和管理, 节点管理器运行在集群的各个计算节点上, 负责管理节点的资源和执行容器。当用户提交计算任务时, 资源管理器根据资源需求和调度策略将任务分配到合适的节点上执行。为保证系统的可扩展性和容错性, YARN 集群通常采用负载均衡机制, 使得计算任务能够均匀分布到各个节点上。

分布式集群是支撑现代云计算与大数据处理的核心基础设施，广泛应用于数据仓库、实时计算、机器学习训练等场景。此类集群的稳定运行直接关系到上层业务的连续性，因此对集群的监控和异常检测具有重要的实际意义。

2.3.2 集群监控数据特点

分布式集群在运行过程中会产生海量的多维度监控时序数据，这些数据具有若干显著特点。

首先是多维度性。集群监控涵盖多个层次和维度，包括系统级指标如 CPU 使用率和内存占用，应用级指标如请求响应时间和连接数，以及集群级指标如任务队列长度和资源使用率等。这些指标从不同角度反映系统的健康状态。

其次是高频采样性。为及时发现异常，监控系统通常以较高频率采集数据，常见的采样间隔为数秒到数分钟。高频采样产生的数据量巨大，对存储和处理提出了较高要求。

第三是多节点相关性。在分布式集群中，由于负载均衡机制的作用，各节点在正常运行时承担的负载相近，其监控指标曲线往往表现出高度的形态相似性。这种相似性为基于群体一致性的异常检测提供了天然的基础。

第四是动态波动性。与用户负载相关的指标具有正常模式难以统一定义的特点，其数值会随系统整体负载发生无规律波动。这种动态性增加了异常检测的难度，使得基于固定阈值或历史模式学习的方法容易产生误报。

2.3.3 分布式异常检测系统架构

随着数据规模的爆炸式增长，分布式集群环境下的时间序列异常检测成为必然趋势。传统的单机异常检测方法在处理大规模、高维度、实时性要求严格的场景时面临严重挑战，包括内存限制、计算瓶颈和可扩展性问题。分布式集群架构通过将计算任务和数据分布到多个节点上，实现了大规模时间序列异常检测的高效处理。

分布式多变量时间序列异常检测系统的架构设计主要遵循三种核心范式：

基于 Spark 的批处理系统架构采用弹性分布式数据集（RDD）作为核心数据抽象，通过内存计算优化大规模时间序列数据的离线分析。Spark 框架的优势在于其强大的内存计算能力和容错机制，通过 DAG 调度器优化计算流程，支持迭代算法的高效执行^[22]。

基于流处理的系统架构（如 Apache Flink）专为实时时间序列异常检测设计，采用数据流作为核心抽象，支持有状态计算和事件时间处理。与基于批处理的 Spark 相比，流处理系统在低延迟流处理方面具有优势，能够满足对实时性要求极高的应用场景。

专门设计的分布式系统则针对特定算法或问题进行深度优化。例如，DADS（分布式异常检测系统）^[36]基于 actor 编程模型，将原始 Series2Graph 算法重构为

反应式协议，通过异步消息传递和分布式计算实现了对单变量时间序列中序列异常的检测。

在数据分区策略方面，时间序列数据分区主要分为时间维度分区和变量维度分区两种模式。时间维度分区将长序列切分为等长的子序列片段，分配给不同计算节点处理。多变量数据的分区策略更为复杂，需要考虑变量间的相关性，基于变量聚类的分区策略有助于在并行计算时保持变量间的语义关联。

2.3.4 异常检测方法在集群数据中的应用与挑战

将现有异常检测方法应用于分布式集群监控数据时面临若干挑战。

第一个挑战是正常模式的定义困难。传统方法假设存在明确的正常模式基线，但对于负载敏感型指标，其正常范围会随集群整体负载动态变化。采用基于历史数据训练的模型进行检测，容易在整体高负载时产生大量误报。

第二个挑战是概念漂移问题。分布式集群的运行模式可能随业务变化、版本升级或配置调整而发生变化，导致历史学习到的模式不再适用于当前数据。传统方法难以区分这种正常的模式演变与真实的异常。

第三个挑战是多节点协同分析的缺失。现有多元时序异常检测方法主要关注时间维度和指标维度，未能有效利用集群中多节点之间的行为相似性。这种相似性蕴含了丰富的先验知识，能够为异常检测提供重要的参考依据。

第四个挑战是计算效率与可扩展性。处理长序列或高维多元序列时，模型中间状态的内存消耗可能呈指数增长，成为性能瓶颈。单机处理能力已无法满足 TB 乃至 PB 级时序数据的分析需求，亟需设计高效的分布式检测算法和系统架构^[36]。

第五个挑战是异常根因定位。检测到异常后，还需定位是哪个或哪几个变量导致了异常，这增加了任务的复杂性，也是当前研究的前沿方向之一。

针对上述挑战，本文提出利用分布式集群多节点曲线的相似性进行异常检测。核心思想是，在负载均衡机制下，正常节点的曲线应保持较高的相似性，而异常节点的曲线会显著偏离群体行为模式。通过对比节点间的曲线差异，能够有效识别离群的异常节点，避免因定义绝对正常而产生的误报问题。

2.4 本章小结

本章系统介绍了时间序列异常检测的相关背景知识与理论基础。首先阐述了时间序列异常数据的基本概念、特征和面临的挑战，并对异常类型进行了系统分类。其次回顾了单变量和多元时序异常检测的主流方法，涵盖了从传统统计方法、经典机器学习方法到深度学习方法的发展脉络，重点介绍了基于重构、预测、图神经网络和因果分析的多元时序异常检测方法。然后介绍了分布式集群的基本概念、监控数据特点和分布式异常检测系统架构，分析了现有异常检测方法在集群场景中应用时面临的挑战。本章内容为后续章节的算法设计奠定了理论基础。

第 3 章 面向多节点单指标的异常检测算法

本章针对分布式集群单指标多节点场景，研究基于曲线相似性的异常检测方法，并提出 CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) 框架。首先，基于联通大数据生产环境构建高质量的单指标多节点时序数据集，为算法研究提供可靠的数据支撑；其次，评估现有多元时序异常检测算法在该数据集上的性能表现，并提出基于欧氏距离、自编码器嵌入和 DBSCAN 聚类的三种曲线相似性度量方法；然后，通过系统的参数敏感性实验揭示方法在实际部署中面临的阈值依赖性和泛化能力不足等核心挑战；最后，提出 CSAD-AT 框架，通过 I-SPOT (Inclusive SPOT) 自适应阈值算法和多视角保守集成决策机制，在保持高检测性能的同时显著降低对人工参数调优的依赖。

3.1 多节点单指标数据集构建

本章研究的数据基础来源于真实的分布式集群生产环境，旨在构建能够反映集群节点协同行为、并包含明确异常标注的数据集，以支撑后续算法的训练、验证与对比分析。

3.1.1 数据来源与采集

本研究的数据来源于中国联通大数据生产底座平台。该平台采用业界标准的 YARN (Yet Another Resource Negotiator) 架构进行分布式计算资源的统一调度与管理，并通过 Prometheus 监控系统对集群运行状态进行全方位、多粒度的实时采集与持久化存储。

在长期运维实践中，我们观察到与用户负载强相关的监控指标（如 RPC 连接数）呈现出独特的统计特性：其正常波动模式难以通过先验知识统一定义，且随全局负载呈现不可预测的剧烈变化。这一特性导致传统基于绝对阈值或历史模式学习的异常检测方法面临高误报率的困境。然而，深入分析发现，得益于分布式集群负载均衡机制的作用，在系统正常运行状态下，同一时刻集群内各节点承担的工作负载趋于均衡，其同一监控指标的时序曲线呈现出高度的形态相似性。这一内在规律性为构建基于“群体行为一致性”的异常检测方法奠定了理论基础。

基于上述洞察，本研究选取“RPC Router 连接数”作为核心研究指标。该指标直接反映数据节点与资源管理器 (ResourceManager) 之间的通信负载状况及连接健康状态，是衡量节点服务能力的关键信号。本研究通过 Prometheus API 接口，系统采集了多个生产集群在连续运行周期内所有节点的该指标时序数据，形成训练集（覆盖 30 天连续运行数据）和测试集（覆盖 8 天连续运行数据），原始数据采样间隔为 60 秒。

3.1.2 数据集预处理与标注

为构建适用于算法研究的高质量数据集，并对后续模型性能进行可靠评估，本研究对从生产环境采集的原始监控时序数据实施了系统化的预处理流程，并采用了半人工标注策略。

原始监控数据在采集与传输过程中，可能因网络延迟、数据采集器 (exporter) 瞬时故障或系统抖动而产生缺失值。为确保时序数据的连续性，本研究采用线性插值法对短暂的数据缺失点进行填充。为确保不同节点间的时序曲线在时间维度上严格对齐，以进行有效的跨节点相似性比较，我们将所有数据点对齐至统一的 60 秒间隔时间戳上。此外，对于因监控代理长期离线、节点重启或下线等原因导致的长时间段数据完全缺失，线性插值将引入较大误差，针对此类情况，本研究直接剔除包含长时间段数据缺失的整条节点记录，以保证进入模型的数据的完整性与可靠性。

获得高质量的异常标注是进行模型评估和算法对比的基础。然而，生产环境中的异常样本缺乏记录，且缺乏自动化标注工具。本研究采用算法初步筛选与专家经验判读相结合的方式半人工标注，旨在确保标注结果的准确性。具体流程如下：

首先，利用多种基础的无监督离群点检测算法对全体时序数据进行初步扫描，且将阈值设为宽松值，筛选出偏离其自身历史基线或群体平均模式的“疑似异常”时间片段。此步骤旨在缩小人工审查的范围，提升标注效率。

其次，对于初筛出的疑似异常片段，关联查询该时段内对应节点的运维数据，如日志、运维工单记录、相关指标时序数据等，对初步筛选出的异常点进行交叉验证，辅助判断其是否为真实的节点级故障或性能异常。

最终，由具备丰富分布式集群运维经验的专家，结合上一步的关联数据，并可视化审查疑似片段及其前后上下文的多节点曲线对比图，最终做出异常点判定。通过以上流程，对两个独立的 YARN 集群测试集进行了标注，形成了包含精确起止时间戳的异常片段集合。

3.1.3 数据集统计与可视化

todo: 修改数据集统计经过预处理与标注，本研究构建了一个适用于分布式集群节点级异常检测研究的单指标多节点时间序列数据集。该数据集的详细统计信息如表3-1所示。

本数据集的核心特征在于其直观揭示了分布式集群在负载均衡机制下的运行规律，如图3-1所示。在绝大部分时间内，集群内多数节点的监控指标曲线呈现出高度的形态同步性与数值聚集性，它们紧密缠绕，形成清晰的“正常行为簇”。这反映了在系统负载均衡策略下，各节点承担相近的工作负载，行为模式高度一致。然而，当个别节点发生资源竞争、进程僵死、网络隔离或本地硬件故障等问题时，其监控曲线会显著偏离这个共同的“行为簇”，表现为形态各异、幅度不一

表 3-1 数据集统计数据

	节点数	数据点数量	异常片段数	异常点数量	异常比例
训练集	30	86400	/	/	/
测试集 A	15	11520	72	901	7.82%
测试集 B	15	11520	33	907	7.87%
测试集	30	23040	105	1808	7.85%

的”离群曲线”。这种”正常聚集，异常离群”的鲜明数据特征，是本研究所提出的、基于群体相似性进行异常检测方法的依据与直观体现。

图 3-1 测试数据集异常标注曲线（红色阴影部分为异常标注）

3.2 现有异常检测方法评估

本节首先系统评估了一系列现有的时间序列异常检测算法在本研究构建的分布式集群数据集上的性能。3.2.1节与3.2.2节分别介绍了选择的先进的异常检测基线算法和基于曲线相似性的检测方法。在3.2.3节介绍了以上算法在分布式集群数据集上的实验结果。这些算法代表了不同的主流范式，其评估结果不仅揭示了现有方法在处理”多节点相似性”场景时的共性局限，也为后续提出的新方法提供了性能对比的锚点。

3.2.1 异常检测基线算法

为全面评估现有方法在分布式集群多节点场景下的适用性，本研究选取了六种具有代表性的先进异常检测算法作为基线，这些算法涵盖了基于重构、基于预测以及基于生成对抗网络等多种主流范式。

在基于重构的方法中，OmniAnomaly 采用随机变分自编码器和平面归一化流来学习多元时间序列的正常模式分布，通过重构概率来识别异常样本。USAD 则在自编码器架构的基础上引入对抗性训练机制，通过增强模型的鲁棒性来提升重构误差对异常的敏感性。

在基于预测的方法中，GDN 通过图神经网络显式建模变量间的拓扑关系，利用对未来值的预测偏差来检测异常。TranAD 基于 Transformer 架构构建深度异常检测模型，借助注意力机制捕捉序列中的长期依赖关系，并通过对抗训练增强对异常的敏感度。MTAD-GAT 结合了图注意力网络和时间卷积网络，能够同时建模变量间的关联关系和时间维度的依赖模式。

在基于生成对抗网络的方法中，MAD-GAN 采用 LSTM 构建生成器和判别器，通过对抗训练学习正常数据的分布特征，综合利用判别器得分和重构误差来识别异常。

实验设置遵循公平对比原则，所有基线模型均使用其作者开源的原版代码和默认推荐参数进行训练与测试。训练阶段使用??节所述的同构集群 30 天正常数据，测试阶段在本研究标注的两个测试集上进行评估并报告合并测试集的整体性能。评估指标采用异常检测领域通用的精确率、召回率和 F1 分数。

3.2.2 基于曲线相似性的异常检测方法

基于前文对分布式集群多节点行为一致性特征的分析，本节选取了三种基于曲线相似性的异常检测方法。这些方法的核心思想是通过度量节点间曲线的差异程度来识别偏离群体行为模式的异常节点，分别从数值距离、深度特征嵌入和密度聚类三个不同角度刻画节点间的相似性关系。

3.2.2.1 基于欧氏距离的数值相似性度量

欧氏距离作为度量空间中最基本的距离度量方式，能够直接量化节点间曲线在数值空间中的差异程度。该方法具有计算简单、物理意义直观的优点，适用于对曲线数值偏离敏感的异常检测场景。

给定一个包含 N 个节点的分布式集群，每个节点 i 在时刻 t 的监控指标观测值记为 x_i^t 。为了捕捉时间序列的局部变化模式并降低瞬时噪声的影响，本方法采用滑动窗口机制对原始序列进行分段处理。对于长度为 w 的时间窗口，节点 i 在时刻 t 的窗口序列可表示为向量形式 $\mathbf{x}_i^t = [x_i^{t-w+1}, x_i^{t-w+2}, \dots, x_i^t]^T \in \mathbb{R}^w$ 。基于此，节点 i 与节点 j 在时刻 t 的欧氏距离定义为两个窗口向量之间的 L2 范数：

$$d_{ij}^t = \|\mathbf{x}_i^t - \mathbf{x}_j^t\|_2 = \sqrt{\sum_{k=t-w+1}^t (x_i^k - x_j^k)^2} \quad (3-1)$$

为了从多节点距离信息中识别异常节点，本方法设计了基于统计标准化的异常分数计算机制。首先，在时刻 t 的滑动窗口内计算所有节点两两之间的欧氏距离，构成距离矩阵 $\mathbf{D}^t \in \mathbb{R}^{N \times N}$ 。然后，计算该距离矩阵中所有非对角元素的均值 μ_t 和标准差 σ_t ，以此表征当前时刻整个集群的曲线整体相似性水平。对于每个节点 i ，计算其与其他所有节点的平均距离：

$$\bar{d}_i^t = \frac{1}{N-1} \sum_{j=1, j \neq i}^N d_{ij}^t \quad (3-2)$$

节点 i 在时刻 t 的异常分数采用 Z-score 标准化形式计算：

$$s_i^t = \frac{\bar{d}_i^t - \mu_t}{\sigma_t} \quad (3-3)$$

该标准化过程将异常分数转换为相对于群体平均水平的偏离程度，消除了不同时刻集群整体负载水平差异带来的影响。在异常判定阶段，设定阈值 T ，若 $s_i^t > T$ ，则判定节点 i 在时刻 t 发生异常。

欧氏距离方法的理论依据在于分布式集群的负载均衡特性。在系统正常运行时,负载均衡机制确保各节点承担相近的工作负载,因此正常节点的监控曲线应保持较高的相似性,其异常分数将聚集在较低水平。当某个节点发生故障或性能异常时,其曲线会显著偏离其他节点的共同模式,导致该节点与其他节点的平均距离增大,异常分数相应升高。

3.2.2.2 基于自编码器嵌入的形态相似性度量

欧氏距离方法对曲线的绝对数值高度敏感,可能忽略形态层面的相似性。例如,两条变化趋势相同但振幅不同的曲线,在欧氏距离度量下差异较大,但形态上实际高度一致。为克服这一局限,本研究引入自编码器进行深度特征提取,通过学习曲线形态的抽象表示来度量节点间相似性。

自编码器是一种无监督表示学习模型,由编码器 f_{enc} 和解码器 f_{dec} 组成。编码器将输入序列 $\mathbf{x}_i^t$ 压缩为低维嵌入向量 $\mathbf{z}_i^t = f_{enc}(\mathbf{x}_i^t)$,解码器从嵌入向量重构原始输入。通过最小化均方重构误差进行训练,自编码器能够在低维空间中保留曲线的本质形态特征,相比原始数值表示,嵌入向量对曲线的整体趋势、周期性和变化模式等形态特征更加敏感,而对绝对数值差异相对不敏感。

在应用阶段,使用编码器为各节点生成嵌入向量,后续基于嵌入向量计算节点间距离 $d_{ij}^t = \|\mathbf{z}_i^t - \mathbf{z}_j^t\|_2$,异常分数计算流程与欧氏距离方法一致。本研究采用的自编码器网络结构包含三层全连接编码器和对称的三层解码中间层维度逐层递减。激活函数采用 ReLU,优化器采用 Adam,学习率设置为 0.001,使用训练集数据训练 100 个 epoch。

3.2.2.3 基于 DBSCAN 聚类的密度相似性度量

除了基于距离度量的方法外,基于密度的聚类算法也能够有效识别偏离主流模式的异常点。DBSCAN 算法是密度聚类的经典代表,其核心思想是将数据空间划分为高密度区域形成的簇和低密度区域中的离群点,具有无需预先指定簇数量、能够发现任意形状簇以及自动识别噪声点等优点。

DBSCAN 算法基于两个关键参数进行聚类:邻域半径 ϵ 和最小样本数 $MinPts$ 。对于数据点 p ,其 ϵ 邻域定义为 $N_\epsilon(p) = \{q \in D | dist(p, q) \leq \epsilon\}$ 。若 $|N_\epsilon(p)| \geq MinPts$,则 p 为核心点;若 p 不是核心点但位于某个核心点的 ϵ 邻域内,则 p 为边界点;否则 p 为噪声点。算法通过密度可达关系将核心点及其密度相连的点聚合为簇,而噪声点则不属于任何簇。

在多节点异常检测场景中,本方法对每个时间窗口内的节点曲线进行 DBSCAN 聚类分析。具体而言,将节点 i 在时刻 t 的窗口序列 $\mathbf{x}_i^t$ 作为特征向量,对所有 N 个节点的特征向量集合执行 DBSCAN 聚类。被算法标记为噪声点的节点即被判定为在该时刻发生异常。

该方法的检测逻辑与分布式集群的运行特性高度契合。在负载均衡机制的作用下,正常运行的节点其监控曲线形态相似,在特征空间中应当聚集形成高密

度簇。而发生故障或性能异常的节点，其曲线会显著偏离主流模式，在特征空间中表现为远离高密度区域的孤立点，从而被 DBSCAN 算法自动识别为噪声。

DBSCAN 方法的主要优势在于无需人工设定异常分数阈值，算法能够自适应地根据数据分布特征划分正常簇和异常点。然而，该方法对邻域半径 ϵ 参数较为敏感，需要根据具体数据特征进行调优。

3.2.3 实验结果对比分析

为验证基于曲线相似性的异常检测方法的有效性，本节在构建的单指标多节点 RPC 连接数数据集上进行系统的对比实验。实验将本文提出的三种方法与多种先进的多元时序异常检测算法进行比较，评估各方法在分布式集群场景下的检测性能。

实验结果如表3-2所示，表中上半部分为六种基线算法的检测结果，下半部分为本文提出的三种基于曲线相似性方法的检测结果。

表 3-2 基线算法与本文方法在 RPC Router 连接数数据集上的异常检测效果对比

算法	Precision	Recall	F1
TranAD	0.8889	0.3932	0.5452
USAD	0.6892	0.5567	0.6159
OmniAnomaly	0.6054	0.4593	0.5224
GDN	0.3101	0.5218	0.3890
MAD_GAN	0.6309	0.6309	0.6891
MTAD_GAT	0.3492	0.1573	0.2169
欧氏距离	0.9826	0.9657	0.9741
自编码器嵌入	0.9792	0.9592	0.9691
DBSCAN	0.9605	0.9659	0.9632

从实验结果可以观察到，现有的多元时序异常检测算法在本数据集上的表现普遍欠佳。具体而言，TranAD 虽然取得了较高的精确率 (0.8889)，但召回率仅为 0.3932，表明该方法遗漏了大量真实异常。USAD 和 OmniAnomaly 作为基于重构的方法，在精确率和召回率之间取得了相对均衡的结果，但 F1 分数均未超过 0.65。GDN 和 MTAD-GAT 作为基于图神经网络的方法，在该数据集上的表现最差，这可能是因为这些方法主要针对多变量间的关联异常设计，而本数据集的异常主要表现为单节点的行为偏离。MAD-GAN 在基线方法中表现相对最好，但 F1 分数也仅为 0.6891。

分析基线算法表现不佳的原因，主要在于负载均衡类指标的固有特性。此类指标的正常波动具有随机性和不可预测性，难以通过历史数据学习出稳定的正常模式。基于重构或预测的深度学习方法往往只能学习到数据的主要趋势，对于负载突变和大幅波动容易产生误报，而对于相对于其他节点偏离但绝对值在正常范围内的异常则容易漏报。

相比之下,本研究使用的三种基于曲线相似性的方法均取得了显著更优的检测效果。欧氏距离方法在最优参数配置下达到了 0.9741 的 F1 分数,精确率和召回率分别为 0.9826 和 0.9657。自编码器嵌入方法取得了 0.9691 的 F1 分数,DBSCAN 方法取得了 0.9632 的 F1 分数。三种方法的 F1 分数均超过 0.96,显著优于所有基线算法。

这一结果充分验证了基于多节点曲线相似性进行异常检测的核心思路的有效性。通过对比节点间的曲线差异而非学习绝对的正常模式,本方法能够有效规避负载敏感型指标波动性强带来的挑战。在负载均衡机制的保证下,正常节点的曲线保持高度相似,而异常节点的曲线则会显著偏离群体模式,这种相对性的判断准则比基于绝对阈值或历史模式的方法更加稳健。

然而,实验也发现三种方法的性能对参数选择较为敏感。欧氏距离和自编码器嵌入方法需要设定合适的异常分数阈值,DBSCAN 方法需要调整邻域半径参数。阈值或参数设置的细微偏差可能导致检测效果急剧下降。针对这一问题,下一节将详细介绍参数敏感性实验分析及相应的优化策略。

3.3 参数敏感性分析与优化策略

3.3.1 阈值敏感性实验与分析

前文提出的三种基于曲线相似性的异常检测方法均涉及关键超参数的设定,这些参数直接影响检测性能的优劣。为深入理解参数选择对算法表现的影响规律,本节在构建的单指标多节点数据集上开展系统的参数敏感性实验,分别考察时间窗口大小和检测阈值两个维度对欧氏距离、自编码器嵌入和 DBSCAN 三种方法检测效果的影响,实验结果如表3-3、表3-4和表3-5所示。

表 3-3 欧氏距离方法在不同阈值和时间窗口下的异常检测分数

threshold	window size=10			window size=20			window size=30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.5485	1.0000	0.7084	0.7251	1.0000	0.8406	0.8214	1.0000	0.9020
3.2	0.6797	1.0000	0.8093	0.7892	1.0000	0.8822	0.8593	1.0000	0.9243
3.4	0.8116	1.0000	0.8960	0.8974	1.0000	0.9459	0.9256	1.0000	0.9614
3.6	0.9532	0.9655	0.9593	0.9826	0.9657	0.9741	0.9727	0.9469	0.9596
3.7	0.9946	0.9242	0.9581	0.9928	0.8839	0.9352	0.9944	0.8856	0.9368
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

表 3-4 AE 嵌入后异常检测分数

threshold	window size=10			window size=20			window size=30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.4370	0.9833	0.6051	0.6028	1.0000	0.7522	0.7206	1.0000	0.8376
3.6	0.9507	0.8977	0.9234	0.9793	0.9404	0.9595	0.9792	0.9592	0.9691
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

表 3-5 DBSCAN 异常检测分数

eps	Precision	Recall	F1
200	0.0332	1.0000	0.0642
300	0.3442	1.0000	0.5121
400	0.8302	1.0000	0.9072
500	0.9605	0.9659	0.9632

从实验结果可以观察到以下规律。在时间窗口维度上，三种方法均表现出窗口越大、检测效果越稳定的趋势。以欧氏距离方法为例，在阈值为 3.4 时，窗口大小从 10 增加到 30，F1 分数从 0.8960 提升至 0.9614。这一现象的原因在于较大的时间窗口能够包含更丰富的时序模式信息，有效平滑瞬时噪声的干扰，使得节点间的相似性度量更加稳健。然而，窗口过大也会导致检测延迟增加，在实际应用中需要在检测精度和响应速度之间进行权衡。

在阈值维度上，三种方法均呈现出典型的精确率与召回率此消彼长的特征。当阈值设置偏低时，算法倾向于将更多的样本判定为异常，导致召回率较高但精确率下降，产生较多误报。当阈值设置偏高时，只有偏离程度极大的样本才会被判定为异常，精确率提高但召回率下降，导致部分真实异常被遗漏。

实验结果进一步揭示了各方法对阈值选择的高度敏感性。以欧氏距离方法为例，当阈值从最优值 3.6 略微调整至 3.7 时，召回率便从 0.9657 骤降至 0.8839，而阈值升至 3.8 时检测完全失效。自编码器嵌入方法表现出类似的敏感特性，最优阈值同样处于一个狭窄的区间内。DBSCAN 方法虽然无需设定异常分数阈值，但其邻域半径参数 ϵ 同样对检测效果影响显著，从 200 到 500 的参数变化导致 F1 分数从 0.0642 跃升至 0.9632，表明参数选择不当会使算法性能产生数量级的差异。

3.3.2 参数依赖性与泛化挑战

上述敏感性实验结果揭示了基于曲线相似性的异常检测方法在实际部署中面临的核心挑战。尽管三种方法在最优参数配置下均能取得优异的检测效果，但这种高性能严重依赖于精确的参数调优，而参数调优过程本身又依赖于充足的标注数据，形成了方法实用化的主要障碍。

从参数依赖性的角度分析，三种方法的最优参数均处于一个狭窄的有效区间内，超出该区间后性能急剧下降。这种“悬崖效应”的根源在于异常检测任务本身的特殊性：异常样本在总体样本中占比极低，导致精确率和召回率对决策边界的位置高度敏感。以欧氏距离方法为例，阈值从 3.6 调整到 3.7 仅变化约 2.8%，但 F1 分数却下降了约 4%，这种非线性的性能衰减特性给参数选择带来了极大的困难。

从泛化能力的角度分析，最优参数的确定严重依赖于特定数据集的统计特性。不同集群的节点数量、负载模式、监控指标量纲等因素均会影响曲线相似性

的分布特征，进而改变最优阈值的位置。在一个集群上调优得到的参数，直接迁移到另一个集群往往难以取得理想效果。这种泛化性不足的问题限制了方法的可扩展性，难以满足大规模分布式系统中多集群统一监控的实际需求。

从工程实践的角度分析，生产环境中获取高质量标注数据的成本极高。异常事件本身具有稀疏性和多样性的特点，完整覆盖各类异常模式需要长期积累。同时，异常标注需要具备领域专业知识的运维人员进行判断，人力成本高昂。更为关键的是，系统运行环境会随时间演化，历史数据上调优的参数可能因概念漂移而逐渐失效，需要持续的人工干预和重新标定。

综合以上分析，如何降低算法对预设阈值的依赖性，实现阈值的动态自适应调整，成为将基于曲线相似性的异常检测方法推向实际生产应用的关键技术挑战。针对这一问题，下一节将引入基于极值理论的自适应阈值算法，并设计多算法集成学习框架，以期在保持高检测性能的同时提升方法的鲁棒性和泛化能力。

3.4 CSAD-AT：基于曲线相似性的自适应阈值异常检测框架

针对前文分析的参数敏感性和泛化能力不足等问题，本节提出 CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) 框架。该框架的核心创新在于将基于曲线相似性的异常检测方法与自适应阈值机制相结合，通过引入改进的极值理论算法实现阈值的动态调整，并设计多视角集成学习策略提升检测的鲁棒性。CSAD-AT 框架的整体架构如图3-2所示，主要包含三个核心模块：多视角相似性度量模块、自适应阈值调整模块和集成决策模块。todo: 结构图

图 3-2 CSAD-AT 框架整体架构

3.4.1 经典 SPOT 算法原理与局限性分析

自适应阈值调整模块的理论基础源于极值理论中的 SPOT (Streaming Peaks Over Threshold) 算法。SPOT 算法的核心思想是利用广义帕累托分布 (Generalized Pareto Distribution, GPD) 对数据分布的尾部进行建模，从而在无需预先假设数据整体分布形式的情况下，自动确定异常判定的阈值边界。

给定时间序列 $\{x_1, x_2, \dots, x_n\}$ ，SPOT 算法首先选取一个初始阈值 t_0 ，将超过该阈值的观测值称为“超额” (exceedance)。根据极值理论中的 Pickands-Balkema-de Haan 定理，当初始阈值 t_0 足够高时，超额值 $Y = X - t_0$ (其中 $X > t_0$) 的条件分布可以用 GPD 来近似：

$$G_{\xi, \sigma}(y) = 1 - \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi}, \quad y > 0 \quad (3-4)$$

其中 ξ 为形状参数， $\sigma > 0$ 为尺度参数。当 $\xi = 0$ 时，GPD 退化为指数分布。

通过对超额值序列进行极大似然估计,可以得到参数 $\hat{\xi}$ 和 $\hat{\sigma}$ 的估计值。进而,对于给定的风险水平 q (通常取 10^{-3} 至 10^{-4}), 异常检测阈值可计算为:

$$z_q = t_0 + \frac{\hat{\sigma}}{\hat{\xi}} \left[\left(\frac{n}{N_t} \cdot q \right)^{-\hat{\xi}} - 1 \right] \quad (3-5)$$

其中 n 为总观测数, N_t 为超过初始阈值 t_0 的观测数。

经典 SPOT 算法在流数据场景下采用滑动窗口机制持续更新 GPD 模型参数,以适应数据分布的动态变化。然而,该算法在更新模型时会判定为异常的数据点排除在训练集之外,这一设计在概念漂移场景下会引发严重的阈值衰减问题。

具体而言,当正常数据的整体分布随时间缓慢上移时,经典 SPOT 算法的更新机制会导致以下恶性循环:首先,由于分布上移,原本处于正常范围边界的数据点开始超过当前阈值,被误判为异常并排除在模型更新之外;其次,用于重新拟合 GPD 的数据池仅包含分布中较低的部分,形成被“截断”的样本集;然后,基于截断样本估计的 GPD 参数会导致新阈值进一步降低;最终,更低的阈值使得更多正常点被误判,进入自我强化的误报螺旋。这一问题在长期运行的生产系统中尤为突出,严重制约了经典 SPOT 算法的实际应用价值。

3.4.2 I-SPOT: 基于包容性极值建模的自适应阈值算法

针对经典 SPOT 算法在概念漂移场景下的阈值衰减问题,本研究提出 I-SPOT (Inclusive SPOT) 算法。该算法的核心创新在于采用包容性极值建模策略,在更新 GPD 模型时将超阈值数据点纳入训练样本,从而使模型能够动态跟踪数据分布的演化趋势。

I-SPOT 算法的设计基于以下关键观察:在分布式集群异常检测场景中,基于曲线相似性计算得到的异常分数序列具有特殊的统计特性。真实的节点级异常通常表现为短暂的尖峰,持续时间有限;而由于系统负载整体变化导致的异常分数上升则呈现渐进式的趋势性变化。因此,超阈值数据点并非全部为真实异常,其中相当比例反映的是正常数据分布的自然演化。将这些数据点简单排除会导致模型对当前数据分布的估计产生系统性偏差。

基于上述分析, I-SPOT 算法对模型更新机制进行了重新设计。设当前时刻的异常分数为 s_t , 当前阈值为 z_t , I-SPOT 算法的更新规则定义如下:

$$D_{t+1} = \begin{cases} D_t \cup \{s_t\}, & \text{if } |D_t| < W_{max} \\ (D_t \setminus \{s_{oldest}\}) \cup \{s_t\}, & \text{otherwise} \end{cases} \quad (3-6)$$

其中 D_t 表示时刻 t 的训练数据池, W_{max} 为数据池的最大容量, s_{oldest} 为数据池中最早的数据点。与经典 SPOT 算法不同, I-SPOT 在更新数据池时不对 s_t 是否超过阈值进行区分,所有观测值均被纳入后续的模式拟合过程。

在阈值更新方面, I-SPOT 算法采用周期性重估策略。每隔 T_{update} 个时间步,算法基于当前数据池 D_t 重新估计 GPD 参数并更新阈值:

$$z_{t+1} = t_0^{(t)} + \frac{\hat{\sigma}^{(t)}}{\hat{\xi}^{(t)}} \left[\left(\frac{|D_t|}{N_t^{(t)}} \cdot q \right)^{-\hat{\xi}^{(t)}} - 1 \right] \quad (3-7)$$

其中上标 (t) 表示基于时刻 t 数据池估计的参数值, $t_0^{(t)}$ 为当前数据池的初始阈值 (通常取数据池的某一高分位数), $N_t^{(t)}$ 为数据池中超过 $t_0^{(t)}$ 的样本数。

通过包容性建模策略, I-SPOT 算法能够在保持对真实异常敏感性的同时, 自适应地跟踪正常数据分布的长期演化趋势。当系统负载整体上升导致异常分数分布右移时, I-SPOT 的阈值会相应上调, 避免产生大量误报; 当系统恢复稳定状态时, 阈值又会逐渐回落至合适水平。这种双向自适应能力使得 I-SPOT 算法特别适用于存在概念漂移的长期运行场景。

在 CSAD-AT 框架中, I-SPOT 算法被应用于多视角相似性度量模块的输出。具体而言, 对于欧氏距离和自编码器嵌入两种方法计算得到的节点平均距离序列, 分别独立运行 I-SPOT 算法进行自适应阈值判定。这种设计使得每种相似性度量方法都能根据其自身的统计特性动态调整决策边界, 提升了框架的整体鲁棒性。

3.4.3 多视角保守集成决策机制

CSAD-AT 框架的第三个核心模块是集成决策模块, 其设计目标是通过融合多种相似性度量方法的检测结果, 进一步提升检测的稳定性和鲁棒性。集成学习的基本思想是利用多个基础模型的互补性, 通过适当的融合策略获得比任何单一模型更可靠的最终判断。

在异常检测任务中, 集成策略的选择需要考虑任务的特殊性。通过对前期实验结果的分析发现, 三种基于曲线相似性的检测方法在宽松阈值设置下均表现出较高的召回率但精确率相对较低的特点, 这表明单一模型更容易产生误报而非漏报。基于这一观察, CSAD-AT 框架采用基于逻辑与的保守集成策略, 其决策规则定义如下:

$$\hat{y}_t = \mathbb{I} \left[\bigwedge_{m=1}^M \left(s_t^{(m)} > z_t^{(m)} \right) \right] \quad (3-8)$$

其中 $\hat{y}_t \in \{0, 1\}$ 表示时刻 t 的最终异常判定结果, M 为基础检测器的数量, $s_t^{(m)}$ 和 $z_t^{(m)}$ 分别表示第 m 个检测器在时刻 t 的异常分数和对应阈值, $\mathbb{I}[\cdot]$ 为指示函数。该策略要求所有基础检测器均判定为异常时, 才将该时刻标记为最终异常。

保守集成策略的理论依据在于不同相似性度量方法对异常模式的敏感性存在差异。欧氏距离方法对数值偏离敏感, 自编码器嵌入方法对形态变化敏感, DBSCAN 方法对密度分布敏感。当某个时刻仅有部分检测器报警时, 可能是由于该检测器对特定类型的正常波动过度敏感所致; 而当所有检测器同时报警时,

则更有可能是真实的异常事件。通过多视角的交叉验证，保守集成策略能够有效过滤单一检测器的误报，提升检测结果的可信度。

在工程实现层面，CSAD-AT 框架对同一时间窗口内的多节点监控数据并行执行三种相似性度量计算，各检测器独立运行 I-SPOT 算法进行自适应阈值判定，最后由集成决策模块汇总各检测器的判定结果并输出最终决策。这种模块化的设计使得框架具有良好的可扩展性，可以方便地引入新的相似性度量方法作为额外的基础检测器。

表 3-6 CSAD-AT 框架保守集成策略的检测效果

检测方法	Precision	Recall	F1
欧氏距离 (threshold=3.0)	0.7251	1.0000	0.8406
自编码器嵌入 (threshold=3.0)	0.6028	1.0000	0.7522
DBSCAN (eps=300)	0.3442	1.0000	0.5121
CSAD-AT 保守集成	0.9053	1.0000	0.9503

3.5 实验验证与对比分析

3.5.1 实验设置

本节实验在本文构建的单指标多节点数据集上进行，旨在验证所提出的改进算法的有效性。实验环境采用配备 Intel Core i7 处理器和 16GB 内存的服务器，算法实现基于 Python 3.8 和相关科学计算库。

实验评估采用异常检测领域通用的三个指标。精确率衡量检测结果中真实异常的比例，计算公式为 $Precision = TP / (TP + FP)$ ，其中 TP 表示正确检测的异常点数量， FP 表示误报的正常点数量。召回率衡量真实异常被正确检测的比例，计算公式为 $Recall = TP / (TP + FN)$ ，其中 FN 表示漏报的异常点数量。F1 分数是精确率和召回率的调和平均，计算公式为 $F1 = 2 \times Precision \times Recall / (Precision + Recall)$ ，用于综合评估检测效果。

实验对比的基线算法包括六种具有代表性的多元时序异常检测算法，涵盖基于重构、预测和生成对抗网络等主流范式。所有基线算法均使用其作者开源的原版代码和默认推荐参数进行训练与测试，以保证对比的公平性。

3.5.2 与基线算法实验结果对比

表3-2展示了各算法在测试集上的检测性能对比。从结果可以看出，传统的多元时序异常检测算法在单指标多节点场景下的表现普遍欠佳。TranAD 算法虽然取得了较高的精确率，但召回率仅为 0.3932，说明该方法漏报了大量异常。MAD_GAN 算法在召回率和精确率之间取得了相对均衡的结果，但 F1 分数仍不足 0.7。MTAD_GAT 算法的表现最差，其原因在于该算法主要针对指标间关联异常设计，难以有效捕捉负载敏感型指标的波动特性。

相比之下,基于曲线相似性的三种方法均取得了显著更优的检测效果。欧氏距离方法在最佳参数设置下达到了 0.9741 的 F1 分数,自编码器嵌入方法达到了 0.9691 的 F1 分数,DBSCAN 方法达到了 0.9632 的 F1 分数。这一结果充分验证了基于多节点曲线相似性进行异常检测的核心思路的有效性。通过对比节点间的曲线差异而非学习绝对的正常模式,能够有效规避负载波动带来的误报问题。

3.5.3 I-SPOT 算法的自适应阈值效果验证

为验证 I-SPOT 算法在阈值自适应方面的有效性,本节设计了对比实验,将基于 I-SPOT 的自适应阈值方法与人工调优的固定阈值方法进行比较。实验中,固定阈值方法采用在验证集上网格搜索得到的最优阈值,而 I-SPOT 方法则完全自动确定阈值,无需任何人工参数调整。实验结果如表3-7所示。

表 3-7 I-SPOT 自适应阈值与固定阈值方法的性能对比

检测方法	Precision	Recall	F1
欧氏距离 (人工调优固定阈值)	0.9826	0.9657	0.9741
欧氏距离 + I-SPOT (自适应阈值)	0.9679	0.9805	0.9742

实验结果表明,I-SPOT 算法在完全无需人工参数调整的情况下,取得了与最优固定阈值方法几乎相同的 F1 分数 (0.9742 vs 0.9741)。值得注意的是,I-SPOT 方法的召回率 (0.9805) 略高于固定阈值方法 (0.9657),表明自适应阈值机制能够更好地捕捉异常模式的动态变化,减少漏报。这一结果验证了 I-SPOT 算法包容性建模策略的有效性:通过将超阈值点纳入模型更新,算法能够动态跟踪数据分布的演化趋势,避免了经典 SPOT 算法在概念漂移场景下的阈值衰减问题。

3.5.4 CSAD-AT 框架的综合性能评估

本节对 CSAD-AT 框架的整体性能进行综合评估,重点考察多视角保守集成策略对检测稳定性的提升效果。如表3-6所示,实验采用宽松阈值设置以模拟实际部署中参数调优不充分的情况,考察集成策略在非最优参数条件下的表现。

实验结果显示,在各基础检测器均采用宽松阈值 (threshold=3.0, eps=300) 的情况下,单一检测器的精确率普遍较低,DBSCAN 方法的精确率仅为 0.3442。然而,通过 CSAD-AT 框架的保守集成策略,最终检测精确率提升至 0.9053, F1 分数达到 0.9503,接近各基础检测器在最优阈值下的表现水平。这一结果充分说明了多视角集成策略的有效性:不同相似性度量方法从数值距离、深度特征和密度分布等不同角度刻画节点间的相似性关系,形成了有效的互补;保守集成策略通过要求多个检测器同时报警,有效过滤了单一检测器的误报,大幅降低了对阈值精确调优的依赖。

综合以上实验结果,CSAD-AT 框架实现了其设计目标:在保持高检测性能的前提下,通过 I-SPOT 自适应阈值算法和多视角保守集成策略,显著降低了方法对人工参数调优的依赖,提升了在实际生产环境中的部署可行性。

3.6 本章小结

本章针对单指标多节点场景下的分布式集群异常检测问题开展了系统研究。首先，基于联通大数据生产环境构建了高质量的单指标多节点时序数据集，采用半人工标注策略确保异常标签的准确性，为算法研究提供了可靠的数据支撑。其次，提出了基于曲线相似性的异常检测方法，分别从数值距离、深度特征嵌入和密度聚类三个视角度量节点间的曲线差异，并通过系统的参数敏感性实验揭示了方法在实际部署中面临的阈值依赖性和泛化能力不足等核心挑战。

针对上述挑战，本章提出了 CSAD-AT (Curve Similarity-based Anomaly Detection with Adaptive Threshold) 框架。该框架的核心创新包括两个方面：一是提出 I-SPOT (Inclusive SPOT) 算法，通过包容性极值建模策略解决了经典 SPOT 算法在概念漂移场景下的阈值衰减问题，实现了阈值的动态自适应调整；二是设计了多视角保守集成决策机制，通过融合多种相似性度量方法的检测结果，有效降低了对单一检测器阈值精确调优的依赖。实验结果表明，CSAD-AT 框架在保持高检测性能的同时，显著提升了方法的鲁棒性和实际部署可行性，为分布式集群的智能运维提供了有效的技术支撑。

第4章 数值-形状双视角融合的多指标多节点异常检测

本章针对分布式集群多指标多节点场景，研究数值与形状双视角融合的异常检测方法。首先介绍所使用的阶跃星辰 GPU 集群故障数据集及其特点；其次提出基于欧氏距离的数值特征异常检测算法；然后设计基于改进 SAX 的形状特征异常检测算法；最后通过加权融合两种视角的检测结果实现故障节点的精准定位，并在真实数据集上验证方法的有效性。

4.1 集群多指标多节点数据集及异常检测算法框架

本节首先对所使用的多指标多节点 GPU 集群异常检测数据集进行介绍，在此基础上分析多指标多节点异常检测问题的关键挑战，并提出数值与形状双视角融合的异常检测算法整体框架。

4.1.1 多指标多节点数据集概述

本节所使用的数据集为阶跃星辰 GPU 计算集群故障数据集，来源于真实生产环境中的大规模 GPU 计算集群监控系统，数据时间跨度从 2024 年 7 月至 2025 年 1 月。

大规模 GPU 集群作为支撑人工智能训练和高性能计算的核心基础设施，其稳定运行对业务连续性至关重要。集群通常由数百台 GPU 计算节点组成，通过统一的资源调度系统执行深度学习训练任务。当集群中某个节点发生故障时，不仅会导致训练任务失败，还可能因日志数据缺失而给后续根因分析带来更大挑战。尽管现有运维工具已能处理大部分常见故障，但对于少数难以定位的故障仍需投入大量人力进行排查，往往需要运维团队和算法团队各领域专家的经验才能解决问题。因此，基于多指标多节点的时间序列数据开展异常检测研究，对于实现 GPU 集群的智能化运维和故障快速定位具有重要的工程价值。

该数据集包含 105 个故障案例作为测试集，同时构造了约 100 个无标签的正常运行作业作为训练集。每个故障案例的根因标注来自运维系统中的专家经验。表4-1统计了测试集中各类故障的分布情况，涵盖 GPU 故障、机器故障、网络故障、PCIe 故障及 NCCL 故障等多种类型。

在 GPU 集群运行过程中，监控系统通过周期性采样的方式持续采集各节点的运行状态数据，形成具有时间对齐关系的多节点多指标时间序列。每个节点会产生多个维度的监控指标，这些指标从不同角度反映节点的运行状态和硬件健康状况。根据指标数值是否随计算任务负载变化，可将监控指标分为负载相关指标和负载无关指标两类。负载无关指标（如 PCIe Width、Network Link Down Event、Cable BER、XID Errors 等）的数值不随计算任务变化，可直接通过与其他机器的对比找到离群节点；负载相关指标（如 GPU 利用率、GPU 温度、显存

表 4-1 故障类别及数量分布

故障类别	数量	典型根因
GPU 故障	51	Double bit error、XID error、GPU 利用率不均衡
机器故障	13	节点宕机、机器检查错误
网络故障	18	交换机问题、光模块故障、线缆故障、信号质量差
PCIe 故障	9	PCIe 性能退化、PCIe 信号质量差
NCCL 故障	5	NCCL 分段错误、NCCL 超时
其他	9	未明确分类的故障

使用率等)则需要更复杂的分析方法来区分正常波动与真实异常。表4-2列出了数据集中的主要监控指标及其含义。

表 4-2 监控指标列表及说明

监控指标	说明
CPU Usage	CPU 繁忙处理任务的时间占比
Memory Usage	系统 RAM 被程序和操作系统占用的比例
GPU PCIe Throughput	GPU 与主板间通过 PCIe 传输数据的速率
NVLink Throughput	通过高速 NVLink 互连传输数据的速率
GPU Temp	GPU 的当前运行温度
Tensor Core Activity	GPU 张量核心用于 AI 计算的利用率
FP32/FP16 Activity	单精度/半精度浮点运算单元的活动百分比
SM Activity	GPU 流式多处理器的利用率
GPU Memory Usage	GPU 专用显存的使用比例
GPU Usage	GPU 整体处理任务的繁忙时间占比
PCIe Width*	PCIe 连接中活动数据通道的数量
Network Link Down Event*	网络接口连接丢失的事件次数
Cable BER*	线缆数据传输过程中的误码率
XID Errors*	GPU 报告的严重错误次数

注：带 * 标记的为负载无关指标。

对于一个包含 N 个节点、 M 个监控指标、时间长度为 T 的数据集，可以形式化表示为一个三维张量：

$$\mathbf{X} \in \mathbb{R}^{N \times M \times T} \quad (4-1)$$

在实际运行环境中，集群节点通常执行相同类型的计算任务，并通过调度系统实现负载均衡，因此在正常运行情况下，各节点在相同指标上的变化趋势通常具有较高的一致性。当某个节点发生异常时，其监控指标曲线往往会偏离这种集群一致性模式。这种偏离可能表现为数值幅度上的异常，例如 GPU 温度显著高

于其他节点；也可能表现为变化模式上的异常，例如功耗曲线出现异常波动或抖动，而数值幅度仍处于正常范围内。这种由系统调度机制和任务一致性所带来的节点间行为一致性，为基于相似性的异常检测方法提供了理论基础。

综上所述，该数据集具有以下特点：(1) 数据来源于真实生产环境，具有较高的工程真实性和复杂性，不同节点之间存在一定程度的自然波动和噪声；(2) 数据具有典型的多指标特征，不同指标对异常的敏感程度不同，仅依赖单一指标难以全面刻画节点异常行为；(3) 数据具有显著的多节点关联特性，在正常运行状态下大多数节点表现出一致的运行模式，而异常节点则表现为相对于其他节点的偏离；(4) 数据具有明显的时间序列特征，异常可能表现为短时突发异常，也可能表现为持续性异常。

4.1.2 问题分析与算法框架

在故障排查实践中存在一种朴素而有效的方法：找到与其他机器表现不同的机器，该机器大概率就是故障节点。基于这一思路，故障节点定位问题可以形式化定义为：给定一个包含 N 台机器的集合 $M = \{m_1, m_2, \dots, m_N\}$ 和一个不相似度函数 $d(\cdot, \cdot)$ ，目标是找到机器 $m^* \in M$ ，使得该机器与其他所有机器的累积不相似度最高，即：

$$m^* = \arg \max_{m_k \in M} D(m_k) \quad (4-2)$$

其中 $D(m_k)$ 为机器 m_k 与集合中其他所有机器的总不相似度。

然而，基于多指标多节点时序数据的故障定位任务面临以下挑战：

高度动态且变化的正常模式。不同规模、不同类型任务的正常运行模式差异显著，传统异常检测方法往往只能识别时序数据中的主要模式，一旦存在其他正常模式变体就可能被误判为异常，导致较高的误报率。

高维度且充满噪声的指标数据。一方面节点的状态需要由多个不同来源的指标共同反映，另一方面这些指标自身因为传感器抖动或采样软件问题产生了大量噪声和抖动，通过这些数据准确刻画正常模式本身就十分困难。

指标敏感度差异与融合策略。不同指标对不同类型故障的敏感程度不同，需要设计合理的多指标融合策略，既能充分利用各指标的判别信息，又能避免单一指标的噪声干扰。

针对上述挑战，本章提出数值与形状双视角融合的异常检测框架，其核心思想是从数值特征和形状特征两个互补视角分析节点异常行为，然后通过加权融合得到最终的异常判定结果。该框架主要包含三个阶段：

数值特征异常检测阶段。采用基于欧氏距离的相似度量方法，通过计算节点间时间序列的数值距离来量化差异程度。对于每个监控指标，计算各节点与其他节点的平均距离作为该指标下的异常分数，然后采用加权平均策略融合多个指标的异常分数，得到数值视角下的综合异常评分。

形状特征异常检测阶段。采用改进的 SAX 符号化算法，通过将时间序列映射为符号序列并计算符号序列间的距离来捕捉曲线形态的差异。该方法取消了

传统 SAX 的分段聚合步骤以保留原始时间分辨率，并采用严格距离计算规则以提升对形状变化的敏感性。

多视角融合与故障定位阶段。将两个视角的异常分数进行加权融合，综合考虑数值偏离和形态变化两方面因素，输出节点的最终异常分数并进行排序。排名靠前的节点即为最可能发生故障的候选节点，供运维人员进一步排查确认。

4.2 基于数值特征的异常检测算法

在分布式集群的性能监控中，节点的多个监控指标在正常状态下往往呈现出与集群整体一致的行为模式。当某个节点发生故障时，其一个或多个监控指标的数值会偏离集群的正常范围，表现为与其他节点的显著差异。基于这一观察，本节提出一种基于数值特征的异常检测算法，其核心思想是：对每个监控指标独立计算节点间的欧氏距离以量化数值差异，通过 Z-score 标准化将距离转换为可比较的异常分数，最后采用加权融合策略整合多指标信息，输出节点的综合异常评分。

4.2.1 基于欧氏距离的单指标异常度量

在多指标多节点场景中，一个直观的思路是将所有指标的时序数据拼接成高维向量，直接计算节点间的多指标联合距离。然而，这种联合计算方式存在以下问题：

量纲与尺度差异。不同监控指标的数值范围和单位存在显著差异，例如 CPU 利用率的取值范围为 0%-100%，而 PCIe 吞吐量可能达到数十 GB/s。直接进行多维欧氏距离计算时，数值尺度较大的指标会主导距离计算结果，导致其他指标的异常信息被掩盖。

故障敏感度差异。不同类型的故障往往只影响部分相关指标，而非所有指标同时异常。例如，GPU 温度异常可能仅导致温度和功耗指标偏离正常范围，而内存使用率和网络吞吐量可能保持正常。若采用多指标联合距离，正常指标的距离贡献会稀释异常指标的判别信号，降低检测灵敏度。

可解释性需求。在实际运维场景中，不仅需要识别异常节点，还需要定位导致异常的具体指标，以指导后续的故障排查。联合距离计算将多指标信息融合为单一数值，丧失了各指标的独立贡献信息。

基于上述考虑，本文采用“先分后合”的策略：首先对每个监控指标独立计算节点间的欧氏距离并转换为异常分数，然后通过加权融合整合多指标的异常信息。这种策略的优势在于：（1）各指标在独立空间中进行距离计算和标准化，避免了量纲差异的干扰；（2）每个指标的异常分数独立可查，便于定位故障相关指标；（3）通过权重调整可以灵活控制各指标对最终结果的贡献程度。

对于单个监控指标 m ，设集群中 N 个节点在时间窗口 $[1, T]$ 内的时序数据为 $\mathbf{X}^{(m)} \in \mathbb{R}^{N \times T}$ ，其中 $\mathbf{x}_i^{(m)} = (x_{i,1}^{(m)}, x_{i,2}^{(m)}, \dots, x_{i,T}^{(m)})$ 表示节点 i 在指标 m 上的时间

序列。节点 i 与节点 j 之间的欧氏距离定义为：

$$d_{ij}^{(m)} = \sqrt{\sum_{t=1}^T (x_{i,t}^{(m)} - x_{j,t}^{(m)})^2} \quad (4-3)$$

通过计算所有节点对之间的距离，构建距离矩阵 $\mathbf{D}^{(m)} \in \mathbb{R}^{N \times N}$ 。节点 i 在指标 m 上的累积距离定义为该节点与其他所有节点距离的平均值：

$$\bar{d}_i^{(m)} = \frac{1}{N-1} \sum_{j \neq i} d_{ij}^{(m)} \quad (4-4)$$

累积距离越大，表明该节点在该指标上与集群整体的偏离程度越高。为了将不同指标的累积距离转换为可比较的异常分数，采用 Z-score 标准化方法：

$$s_i^{(m)} = \frac{\bar{d}_i^{(m)} - \mu_d^{(m)}}{\sigma_d^{(m)}} \quad (4-5)$$

其中 $\mu_d^{(m)}$ 和 $\sigma_d^{(m)}$ 分别为所有节点累积距离的均值和标准差。标准化后的异常分数 $s_i^{(m)}$ 表示节点 i 在指标 m 上的累积距离相对于集群平均水平的偏离程度，正值越大表示异常程度越高。

4.2.2 多指标加权融合策略

在获得各指标的独立异常分数后，需要将其融合为节点的综合异常评分。本文采用加权求和的融合策略，节点 i 的综合异常分数定义为：

$$S_i = \sum_{m=1}^M w_m \cdot \max(s_i^{(m)}, 0) \quad (4-6)$$

其中 w_m 为指标 m 的权重，满足 $\sum_{m=1}^M w_m = 1$ 。

公式4-6中对异常分数取 $\max(\cdot, 0)$ 操作的原因如下：当 $s_i^{(m)} < 0$ 时，表示节点 i 在指标 m 上的累积距离低于集群平均水平，即该节点在该指标上表现正常甚至优于平均。在故障检测场景中，这种“正常”信号不应对最终的异常判定产生负面影响，因此将负值截断为 0。此外，当某指标的异常分数为 0 时，意味着该指标在当前时间段未能提供有效的异常判别信息，在融合时自然不产生贡献。

权重的设定可以采用以下策略：

等权重策略。在缺乏先验知识的情况下，对所有指标赋予相等权重，即 $w_m = 1/M$ 。这种策略简单直观，适用于探索性分析阶段。

基于领域知识的权重。根据运维经验，对关键性指标（如 GPU 利用率、温度、功耗等）赋予较高权重，对辅助性指标赋予较低权重。这种策略能够突出重要指标的判别作用。

基于历史数据的自适应权重。通过分析历史故障案例中各指标的检测效果，自动学习最优权重配置。例如，可以采用基于验证集性能的网格搜索或贝叶斯优化方法确定权重。

在本文的实验中，默认采用等权重策略，并在消融实验中验证不同权重设置对检测性能的影响。

4.2.3 算法实现与效率优化

算法1给出了基于数值特征的异常检测算法的完整流程。算法输入为多节点多指标时序数据张量 $\mathbf{X} \in \mathbb{R}^{N \times M \times T}$ 和指标权重向量 $\mathbf{w}$ ，输出为各节点的综合异常分数 $\mathbf{S}$ 。

算法 1 基于数值特征的多指标异常检测算法

Require: 时序数据 $\mathbf{X} \in \mathbb{R}^{N \times M \times T}$ ，权重向量 $\mathbf{w} \in \mathbb{R}^M$

Ensure: 异常分数向量 $\mathbf{S} \in \mathbb{R}^N$

- 1: 初始化 $\mathbf{S} \leftarrow \mathbf{0}$
 - 2: **for** $m = 1$ to M **do** ▷ 遍历每个指标
 - 3: $\mathbf{D}^{(m)} \leftarrow$ 计算节点间欧氏距离矩阵 ▷ $\mathbf{D}^{(m)} \in \mathbb{R}^{N \times N}$
 - 4: $\bar{\mathbf{d}}^{(m)} \leftarrow$ 计算各节点累积距离 ▷ $\bar{d}_i^{(m)} = \frac{1}{N-1} \sum_{j \neq i} D_{ij}^{(m)}$
 - 5: $\mathbf{s}^{(m)} \leftarrow$ Z-score 标准化 ($\bar{\mathbf{d}}^{(m)}$) ▷ 转换为异常分数
 - 6: $\mathbf{S} \leftarrow \mathbf{S} + w_m \cdot \max(\mathbf{s}^{(m)}, 0)$ ▷ 加权累加
 - 7: **end for**
 - 8: **return** $\mathbf{S}$
-

该算法的时间复杂度为 $O(M \cdot N^2 \cdot T)$ ，其中距离矩阵计算是主要的计算开销。为满足大规模集群实时监控的需求，本文在算法实现层面进行了以下效率优化：

向量化距离计算。采用 NumPy 和 SciPy 库的向量化计算功能，将循环操作转换为矩阵运算。具体地，利用 `scipy.spatial.distance.cdist` 函数一次性计算整个距离矩阵，避免显式的双重循环，充分利用底层优化的线性代数库（如 BLAS）的并行计算能力。

数据预处理。对监控数据进行预处理以确保数据质量：采用线性插值填充缺失值，确保时序数据的完整性；按时间窗口切分数据，将长时序分解为可独立处理的短窗口，降低单次计算的内存占用。

增量计算与缓存。对于滑动窗口检测场景，采用增量更新策略：当新数据到达时，仅重新计算涉及新数据点的距离，而非完全重建距离矩阵。同时缓存中间计算结果（如各节点的历史累积距离统计量），减少重复计算。

通过上述优化措施，算法在处理包含 50 个节点、6 个监控指标、1000 个时间点的典型数据规模时，单次检测耗时可控制在 0.3 秒以内，满足秒级响应的实时监控需求。

4.3 基于形状特征的异常检测算法

4.3.1 经典 SAX 算法原理

SAX 是一种经典的时间序列符号化表示方法，其核心思想是将连续的时间序列转换为离散的符号序列，从而实现时间序列的降维和模式匹配。传统 SAX 算法主要包括四个步骤。

第一步是归一化处理。将原始时间序列标准化为均值为 0、标准差为 1 的标准正态分布，以消除量纲和基线差异。对于时间序列 $\{x_1, x_2, \dots, x_T\}$ ，归一化后的序列为：

$$\hat{x}_t = \frac{x_t - \mu}{\sigma} \quad (4-7)$$

其中 μ 和 σ 分别为序列的均值和标准差。

第二步是分段聚合近似。将归一化后的时间序列划分为 w 个等长的片段，并计算每个片段内所有数据点的平均值作为该片段的代表值。这一步骤实现了序列的降维，将长度为 T 的序列压缩为长度为 w 的表示。

第三步是符号化映射。基于标准正态分布的分位数点将数值空间划分为 α 个等概率的区间，其中 α 为字母表的大小。每个片段的平均值根据其所落入的区间被映射到对应的符号。通过这种方式，连续的时间序列被转换为由 α 个符号组成的符号序列。

第四步是距离计算。在符号空间中定义符号间的距离，进而计算符号序列之间的距离。传统 SAX 距离的定义中，相邻符号之间的距离为 0，非相邻符号之间的距离为对应分位数点之差。

4.3.2 基于 SAX 算法的符号化表示与模式匹配

为适应 GPU 集群故障检测场景的特殊需求，本文对传统 SAX 算法进行了两方面改进。

第一个改进是取消分段聚合步骤，保留原始时间分辨率。传统 SAX 的分段聚合虽然实现了降维，但可能丢失重要的细节信息。在 GPU 性能监控场景中，瞬时尖峰和短暂抖动往往包含重要的故障指示信息。因此，本文取消了分段处理，对归一化后的每个数据点直接进行符号化，使算法能够捕捉到更细微的时间点异常。

第二个改进是采用严格距离计算规则。传统 SAX 算法中，相邻符号被视为相似，其距离设为 0。在故障检测场景中，这种宽容性可能导致对形状变化的不敏感而造成漏报。本文修改距离计算规则，仅当两个符号完全相同时距离才为 0，否则计算实际距离：

$$d'(l_i, l_j) = \begin{cases} 0, & \text{if } i = j \\ \beta_{\max(i,j)-1} - \beta_{\min(i,j)}, & \text{if } i \neq j \end{cases} \quad (4-8)$$

其中 β_k 为第 k 个高斯分位数点。

基于改进的 SAX 算法，多节点形状异常检测的流程如下。首先对各节点的时间序列进行归一化处理，然后根据预设字母表大小计算高斯分位数点并将归一化序列映射为符号序列。接着计算所有节点对之间的符号序列距离，构建节点距离矩阵。最后计算每个节点与其他节点的平均距离，基于 Z-score 方法计算异常分数，并对各指标的异常分数进行加权聚合得到最终结果。

4.3.3 算法效率优化

为满足大规模 GPU 集群实时监控的需求，本文在算法实现层面进行了效率优化。

在距离矩阵计算方面，采用预计算符号距离表和向量化操作来避免运行时的重复计算。对于字母表大小为 α 的 SAX 表示，预先计算并存储 $\alpha \times \alpha$ 的符号距离矩阵，在运行时通过查表方式获取距离值。同时利用 NumPy 和 SciPy 库的向量化计算功能，将循环操作转换为矩阵运算，充分利用底层优化的线性代数库。

在数据处理方面，采用批量处理策略和内存优化技术。对于大规模数据集，按时间窗口进行分批处理，避免一次性加载全部数据导致的内存压力。同时采用稀疏矩阵表示和即时计算策略，仅在需要时计算特定节点对之间的距离，减少不必要的计算开销。

通过上述优化措施，算法在处理包含数百个节点、数千个时间点的多指标数据集时，单次检测耗时能够控制在秒级，满足实时监控的响应要求。

4.4 多算法融合与故障定位

4.4.1 加权融合设计

在多算法融合阶段，需要将基于数值特征和基于形状特征的检测结果进行综合。本文采用加权平均的融合策略，具体设计如下。

设节点 i 通过数值检测算法得到的异常分数为 s_i^{num} ，通过形状检测算法得到的异常分数为 s_i^{shape} 。由于两种算法输出的分数尺度可能不同，首先对各自的分数进行归一化处理，使其落入可比较的范围。然后通过加权平均得到融合后的异常分数：

$$s_i^{final} = w_{num} \cdot s_i^{num} + w_{shape} \cdot s_i^{shape} \quad (4-9)$$

其中 w_{num} 和 w_{shape} 分别为数值检测和形状检测的权重，满足 $w_{num} + w_{shape} = 1$ 。

权重的设置可以基于领域知识或历史检测性能进行调整。在默认情况下，本文采用等权重设置，即 $w_{num} = w_{shape} = 0.5$ 。在实际应用中，如果已知某类故障更多表现为数值异常，可以适当提高数值检测的权重，反之亦然。

对于多指标场景，在单一视角内部也需要进行多指标融合。本文采用的策略是对各指标分别计算异常分数，然后进行加权聚合。在加权聚合时，遵循以下规则以提高鲁棒性。异常分数为 0 的指标不参与聚合计算，因为这表明该指标在当前时间段内未能提供有效的异常信息。异常分数为负数的指标按 0 计算，因为负

的 Z-score 意味着该节点在该指标上表现正常甚至优于平均水平，不对最终的异常判定产生负面影响。

4.4.2 故障定位与根因分析建议

在获得各节点的融合异常分数后，根据分数对节点进行降序排序，排名靠前的节点即为最可能发生故障的节点。在实际应用中，通常取 Top-K 个节点作为故障候选进行进一步排查。

基于数值和形状双视角的检测结果，还可以为故障类型判断提供参考。当节点表现出高数值异常分数且高形状异常分数时，可能对应严重的硬件故障或系统级问题，例如 GPU 核心损坏或散热系统失效。当节点表现出高数值异常分数但低形状异常分数时，可能对应数值漂移或配置异常，例如功耗阈值设置不当或资源分配失衡。当节点表现出低数值异常分数但高形状异常分数时，可能对应间歇性故障或软件异常，例如驱动程序问题或进程异常。

这种故障类型的初步判断能够为运维人员的后续排查工作提供方向性指导，提高故障定位的效率。

4.5 实验验证与对比分析

4.5.1 实验设置

本节实验在阶跃星辰 GPU 计算集群故障数据集上进行。该数据集包含 105 个真实故障案例，每个案例对应一次明确记录的 GPU 节点故障事件，故障类型涵盖 GPU 性能退化、温度异常升高、功耗异常波动等。在每个故障案例中，数据集包含故障节点及其所在集群所有节点在故障发生时间窗口内的多指标时间序列数据。

实验评估采用 Acc@5 和 Avg Rank 两个指标。Acc@5 表示真实故障机器出现在算法检测 Top-5 中的案例比例，衡量算法的故障检测准确率。Avg Rank 为真实故障机器在 Top-5 中的平均排名，衡量算法的故障定位效率，数值越小表示定位越精准。

基线算法选择马氏距离方法，该方法考虑了变量之间的协方差关系，是多元异常检测领域的经典方法。

4.5.2 方法结果对比分析与案例研究

表4-3展示了各方法在 105 个故障案例上的检测效果对比。

从结果可以看出，基于欧氏距离的数值检测方法在 Acc@5 指标上达到 0.800，显著优于马氏距离基线方法的 0.453。这表明在 GPU 集群场景中，基于节点间曲线相似性的检测思路比基于变量协方差关系的传统方法更为有效。

基于改进 SAX 的形状检测方法取得了 0.762 的 Acc@5，虽然略低于数值检测方法，但能够捕捉到一些数值方法遗漏的形态异常案例。将两种方法进行融合

表 4-3 多指标多节点异常检测方法对比

算法	Acc@5	Avg Rank
马氏距离	0.453	1.20
欧氏距离 + 加权聚合	0.800	1.25
改进 SAX+ 加权聚合	0.762	1.30
数值-形状融合	0.838	1.18

后，Acc@5 提升至 0.838，Avg Rank 降至 1.18，验证了双视角融合策略的有效性。

4.5.3 融合方法优势分析

融合方法相比单一视角方法具有两方面优势。

第一是更全面的异常覆盖。数值检测方法擅长识别幅度显著偏离的异常，而形状检测方法擅长识别变化模式异常但幅度正常的情况。通过融合两种视角，能够覆盖更多类型的异常模式。实验统计表明，约 15% 的故障案例主要表现为形态异常而非数值异常，单纯依赖数值方法会造成漏报。

第二是更稳定的检测性能。单一方法可能在特定类型的故障上表现优异但在其他类型上效果欠佳，而融合方法通过综合多个视角的判断结果，能够获得更稳定的整体性能，降低了对特定故障类型的依赖。

4.5.4 算法效率实验对比

为验证算法效率优化的效果，本节对比了优化前后的算法执行时间。实验选取包含 50 个节点、6 个监控指标、1000 个时间点的典型数据规模进行测试。

表 4-4 算法效率对比

算法	平均执行时间	加速比
欧氏距离（优化前）	2.35s	1.0×
欧氏距离（优化后）	0.28s	8.4×
改进 SAX（优化前）	5.12s	1.0×
改进 SAX（优化后）	0.45s	11.4×

通过向量化计算和预计算优化，欧氏距离方法的执行时间从 2.35 秒降至 0.28 秒，加速比达到 8.4 倍。改进 SAX 方法的执行时间从 5.12 秒降至 0.45 秒，加速比达到 11.4 倍。优化后的算法能够满足实时监控场景的响应要求。

4.6 本章小结

本章针对多指标多节点场景下的 GPU 集群异常检测问题，提出了数值与形状双视角融合的异常检测框架。在数值视角方面，采用基于欧氏距离的多指标加

权融合方法量化节点间的数值差异。在形状视角方面，提出了改进的 SAX 符号化表示算法，通过保留原始时间分辨率和采用严格距离计算规则来提升对形状变化的敏感性。最后通过加权融合两种视角的检测结果，实现了对不同类型异常的全面覆盖。在真实 GPU 集群故障数据集上的实验表明，所提融合方法的故障检测准确率达到 83.8%，显著优于单一视角方法和传统基线方法。

第 5 章 基于曲线相似性的异常检测系统

在前述章节中，本文分别针对单指标多节点和多指标多节点两种典型场景，提出了基于曲线相似性的异常检测方法，并通过实验验证了其有效性。然而，算法层面的研究成果要真正服务于生产环境，还需要构建一个完整的系统平台，将数据采集、预处理、异常检测、告警通知和可视化分析等环节有机串联。本章聚焦于将上述研究成果转化为可落地的工程系统，详细介绍基于曲线相似性的分布式集群异常检测系统的设计与实现。系统以联通大数据生产底座平台为应用背景，对接 Prometheus 监控体系实现常态化巡检，并构建了完整的多级告警机制，旨在为运维团队提供一套自动化、智能化的集群健康监控解决方案。

5.1 系统设计

5.1.1 功能模块设计

基于曲线相似性的异常检测系统旨在为分布式集群提供自动化的节点级异常监控与告警能力。根据实际生产环境的运维需求，系统采用模块化设计思想，划分为七个核心功能模块：用户与权限管理模块、数据接入与管理模块、数据预处理模块、检测引擎模块、常态化监控模块、告警管理模块以及可视化与数据看板模块。各模块之间通过标准化接口实现数据的无缝流转与高效交互，确保异常检测的及时性与准确性。

用户与权限管理模块负责系统用户的身份验证与权限控制，确保不同角色的用户仅能访问其授权范围内的数据和功能。该模块支持用户注册与登录、角色分配（管理员和普通用户）以及操作日志记录等功能，为多用户协同运维提供安全的管理基础。

数据接入与管理模块负责对接多种数据源，实现监控数据的统一采集与管理。该模块支持两种数据输入方式：一是通过文件上传导入 CSV 或 JSON 格式的历史数据，适用于离线分析和算法验证场景；二是通过 Prometheus API 接口实时采集集群监控指标，适用于在线监控场景。模块内置数据集管理功能，支持数据集的查询、预览和删除操作，并提供时序数据的图表化预览能力。

数据预处理模块针对采集到的原始监控数据进行系统性清洗与转化，主要功能包括时间戳对齐、缺失值填充、数据去噪和归一化处理。该模块采用流水线式的处理架构，将各预处理步骤串联执行，确保输出数据在时间维度上对齐、在数值维度上可比，为后续检测算法提供高质量的输入数据。

检测引擎模块是系统的核心计算组件，封装了本文提出的全部异常检测算法。对于单指标多节点场景，支持欧氏距离检测、自编码器嵌入检测和 DBSCAN 聚类检测三种方法，并通过 I-SPOT 自适应阈值和保守集成决策实现鲁棒的异常判定。对于多指标多节点场景，支持数值检测与形状检测的双视角融合框架。检

测引擎采用统一的调度接口，根据用户配置的场景类型和算法参数自动编排检测流程。

常态化监控模块是系统区别于一次性离线分析工具的关键特性。该模块通过定时任务调度机制，按照预设的巡检周期自动从 Prometheus 拉取最新的监控数据，执行预处理和异常检测流程，并将检测结果写入数据库。运维人员无需手动触发检测任务，系统即可实现对集群健康状态的 7×24 小时持续监控。

告警管理模块基于检测引擎输出的异常评分，实现分级告警与多渠道通知。系统定义了三个告警级别：信息级（异常分数轻微偏高，记录但不主动推送）、警告级（异常分数显著偏高，需要运维人员关注）和严重级（异常分数极高，需要立即处置）。当检测到超过预设阈值的异常事件时，系统自动生成告警记录并通过邮件、企业微信等渠道向相关人员推送告警通知。模块同时提供告警日志的查询、筛选和统计分析功能，支持告警策略的灵活配置。

可视化与数据看板模块通过交互式图表和数据面板，为运维人员提供全方位的集群健康状态视图。主要功能包括实时数据监控大屏、多节点曲线对比分析、异常分数排名展示、时间线热力图、历史检测结果回溯以及告警统计报表等，帮助运维人员快速定位异常节点并分析异常原因。

5.1.2 系统架构设计

系统采用前后端分离的分层架构设计，从上至下分为用户层、应用层、算法层和数据层四个层次，各层之间职责明确、接口规范，便于独立开发和水平扩展。

用户层面向最终使用者，提供基于 Web 浏览器的可视化交互界面。前端基于 HTML5、CSS3 和 JavaScript 构建，利用 ECharts 图表库实现丰富的数据可视化效果，通过 AJAX 异步请求与后端 API 进行数据交互。用户层支持响应式布局，可在不同分辨率的终端设备上正常访问。

应用层承接用户请求并协调各功能模块的工作，是连接用户界面与核心算法的桥梁。后端基于 Python 的 Flask 轻量级 Web 框架构建 RESTful API 服务，负责路由分发、请求参数校验、业务逻辑编排和响应结果封装。应用层同时负责定时任务的调度管理，通过 APScheduler 框架实现常态化监控任务的定时触发。此外，应用层还集成了告警通知服务，通过 SMTP 协议发送邮件告警，通过 Webhook 接口对接企业微信等即时通讯工具。

算法层封装了本文提出的全部异常检测算法，是系统的核心计算引擎。算法层接收应用层传递的标准化数据和配置参数，调用相应的检测算法进行计算，并将异常评分和检测结论返回给应用层。算法层基于 Python 科学计算生态构建，核心依赖包括 NumPy、SciPy 和 scikit-learn 等数值计算库，自编码器模块基于 PyTorch 深度学习框架实现。

数据层负责系统全部数据的持久化存储与高效检索。针对不同类型数据的存储需求，系统采用关系型数据库与缓存相结合的策略：使用 SQLite 关系型数

数据库存储用户信息、集群配置、检测任务、检测结果、告警规则和告警记录等结构化数据；使用基于内存的 LRU 缓存机制存储实时检测结果和高频访问数据，减轻数据库查询压力，提高系统响应速度。对于 Prometheus 时序数据，系统不做本地持久化存储，而是在每次检测时通过 API 实时拉取，避免数据冗余和同步问题。

5.1.3 数据库设计

系统的结构化数据采用 SQLite 关系型数据库进行存储，共设计了六张核心数据表，覆盖用户管理、集群配置、检测任务、检测结果、告警规则和告警记录等关键业务实体。各表之间通过主外键关联，保障数据的完整性与一致性。

(1) 用户表 (users)

用户表存储系统用户的基础信息和角色权限，用于实现安全的身份验证与访问控制。具体表结构如表5-1所示。

表 5-1 用户表

字段名	类型	说明	约束
user_id	INTEGER	用户 ID (主键)	PRIMARY KEY AUTOINCREMENT
username	VARCHAR(50)	用户名	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	密码哈希值	NOT NULL
role	VARCHAR(20)	用户角色	DEFAULT 'user'
email	VARCHAR(100)	邮箱地址	
created_at	DATETIME	创建时间	DEFAULT CURRENT_TIMESTAMP

(2) 集群配置表 (cluster_configs)

集群配置表存储被监控集群的基本信息和 Prometheus 数据源配置，作为常态化监控任务的配置基础。具体表结构如表5-2所示。

(3) 检测任务表 (detection_tasks)

检测任务表记录每次异常检测的执行信息，包括手动触发和定时巡检两种来源的检测任务。具体表结构如表5-3所示。

其中，task_type 字段区分手动检测 (manual) 和定时巡检 (scheduled) 两种任务来源；status 字段记录任务的执行状态，取值包括 pending (等待中)、running (执行中)、completed (已完成) 和 failed (失败) 四种状态。

(4) 检测结果表 (detection_results)

检测结果表存储每次异常检测的节点级结果数据，包括异常评分、是否异常等信息。具体表结构如表5-4所示。

(5) 告警规则表 (alert_rules)

表 5-2 集群配置表

字段名	类型	说明	约束
cluster_id	INTEGER	集群 ID (主键)	PRIMARY KEY AUTOINCREMENT
cluster_name	VARCHAR(100)	集群名称	NOT NULL
cluster_type	VARCHAR(50)	集群类型	NOT NULL
prometheus_url	VARCHAR(255)	Prometheus 地址	NOT NULL
promql_query	TEXT	PromQL 查询表达式	NOT NULL
check_interval	INTEGER	巡检间隔 (分钟)	DEFAULT 5
scenario_type	VARCHAR(20)	检测场景类型	NOT NULL
is_active	BOOLEAN	是否启用	DEFAULT TRUE
created_by	INTEGER	创建者 ID	FOREIGN KEY(users)

表 5-3 检测任务表

字段名	类型	说明	约束
task_id	INTEGER	任务 ID (主键)	PRIMARY KEY AUTOINCREMENT
cluster_id	INTEGER	关联集群 ID	FOREIGN KEY(cluster_configs)
task_type	VARCHAR(20)	任务类型	NOT NULL
scenario	VARCHAR(20)	检测场景	NOT NULL
algorithm_config	TEXT	算法配置 (JSON)	
status	VARCHAR(20)	任务状态	DEFAULT 'pending'
started_at	DATETIME	开始时间	
finished_at	DATETIME	结束时间	

表 5-4 检测结果表

字段名	类型	说明	约束
result_id	INTEGER	结果 ID (主键)	PRIMARY KEY AUTOINCREMENT
task_id	INTEGER	关联任务 ID	FOREIGN KEY(detection_tasks)
node_name	VARCHAR(100)	节点名称	NOT NULL
anomaly_score	FLOAT	异常评分	NOT NULL
is_anomaly	BOOLEAN	是否异常	NOT NULL
detection_time	DATETIME	检测时间	NOT NULL
details	TEXT	详细信息 (JSON)	

告警规则表存储用户配置的告警策略，定义不同级别告警的触发条件和通知方式。具体表结构如表5-5所示。

表 5-5 告警规则表

字段名	类型	说明	约束
rule_id	INTEGER	规则 ID（主键）	PRIMARY KEY AUTOINCREMENT
cluster_id	INTEGER	关联集群 ID	FOREIGN KEY(cluster_configs)
alert_level	INTEGER	告警级别	NOT NULL
score_threshold	FLOAT	分数阈值	NOT NULL
notify_method	VARCHAR(50)	通知方式	NOT NULL
notify_target	VARCHAR(255)	通知目标	NOT NULL
is_enabled	BOOLEAN	是否启用	DEFAULT TRUE
created_at	DATETIME	创建时间	DEFAULT CURRENT_TIMESTAMP

其中，alert_level 字段定义了三个告警级别：1 表示信息级，2 表示警告级，3 表示严重级。notify_method 字段支持 email（邮件通知）、webhook（企业微信/钉钉 Webhook）和 system（仅系统内记录）三种通知方式。

(6) 告警记录表（alert_logs）

告警记录表保存系统触发的所有告警事件及其处理状态，便于后续追踪和统计分析。具体表结构如表5-6所示。

表 5-6 告警记录表

字段名	类型	说明	约束
log_id	INTEGER	日志 ID（主键）	PRIMARY KEY AUTOINCREMENT
task_id	INTEGER	关联任务 ID	FOREIGN KEY(detection_tasks)
alert_level	INTEGER	告警级别	NOT NULL
node_name	VARCHAR(100)	异常节点	NOT NULL
anomaly_score	FLOAT	异常评分	NOT NULL
alert_message	TEXT	告警信息	NOT NULL
notify_status	VARCHAR(20)	通知状态	DEFAULT 'pending'
acknowledged	BOOLEAN	是否已确认	DEFAULT FALSE
alert_time	DATETIME	告警时间	NOT NULL

5.2 开发环境与技术选型

系统采用前后端分离的开发模式，结合多种成熟的技术框架和工具库，确保系统在异常检测任务中的高效性和稳定性。

在后端技术方面，系统基于 Python 3.9 开发环境构建，选用 Flask 2.3 作为 Web 服务框架，提供轻量级的 RESTful API 服务能力。核心异常检测算法依赖 NumPy 1.24、SciPy 1.10 和 scikit-learn 1.2 等科学计算库实现高效的数值运算，自编码器模块基于 PyTorch 2.0 深度学习框架构建。定时任务调度采用 APScheduler 3.10 框架，支持基于 Cron 表达式的灵活调度配置。数据处理依赖 Pandas 2.0 进行时序数据的清洗与转换。告警通知模块通过 Python 标准库 smtplib 实现邮件发送，通过 requests 库对接企业微信 Webhook 接口。

在前端技术方面，界面基于 HTML5、CSS3 和 JavaScript 构建，采用 ECharts 5.4 图表库实现丰富的交互式数据可视化效果，包括折线图、柱状图、热力图和仪表盘等多种图表类型。页面布局采用响应式设计，通过 CSS Flexbox 和 Grid 布局实现自适应的多终端访问。前端与后端之间通过 Fetch API 进行异步数据通信。

在数据存储方面，结构化数据采用 SQLite 3 关系型数据库存储，适合中等规模的部署场景，无需额外的数据库服务进程。系统同时在应用层实现了基于 Python 字典和 functools.lru_cache 的内存缓存机制，用于缓存高频访问的检测结果和配置数据，减少数据库 IO 开销。

系统的开发和测试环境配置如下：操作系统为 Ubuntu 20.04 LTS，CPU 为 Intel Xeon E5-2680 v4，内存为 64GB DDR4，GPU 为 NVIDIA Tesla V100（用于自编码器模型训练）。生产环境部署于联通大数据平台的容器化基础设施上，通过 Docker 容器实现服务的隔离部署和弹性伸缩。系统的代码组织结构如表5-7所示。

表 5-7 系统代码目录结构

目录/文件	功能说明
app.py	Flask 主应用，路由定义与 API 接口
config.py	系统配置与算法默认参数
models.py	数据库 ORM 模型定义
auth.py	用户认证与权限控制
algorithms/	核心算法模块
euclidean.py	欧氏距离相似性检测
autoencoder.py	自编码器嵌入检测
dbscan_detector.py	DBSCAN 聚类检测
ispot.py	I-SPOT 自适应阈值
ensemble.py	保守集成决策
sax_improved.py	改进 SAX 形状检测
fusion.py	双视角融合
services/	业务服务模块
data_loader.py	数据加载（文件上传/API）
preprocessor.py	预处理流水线
detection_engine.py	检测引擎调度
scheduler.py	常态化监控定时调度
alert_service.py	告警规则匹配与通知分发
templates/	前端 HTML 模板
static/	静态资源（CSS/JS/图表组件）

5.3 系统实现

系统具体实现的功能包含用户与权限管理、数据接入与管理、数据预处理、检测引擎、常态化监控、告警管理以及可视化与数据看板七个模块。以下分别介绍各模块的实现细节。

5.3.1 用户与权限管理模块

用户与权限管理模块为系统提供安全的身份认证和细粒度的访问控制能力。系统设计了独立的登录注册页面，用户输入用户名和密码后，后端对密码进行 SHA-256 哈希校验，验证通过后生成基于 Flask-Session 的会话令牌，后续请求通过 Cookie 携带会话信息实现身份保持。

系统定义了管理员 (admin) 和普通用户 (user) 两种角色。管理员拥有系统的全部操作权限，包括用户管理、集群配置、告警规则设定和系统参数调整等；普通用户可以使用数据上传、手动检测、结果查看和告警日志查询等常规功能。权限控制通过 Flask 的装饰器机制实现，在每个受保护的 API 接口上添加角色检查逻辑，确保越权请求被及时拦截。

管理员登录后，可以在用户管理页面中查看所有注册用户的信息列表，包括用户名、角色、邮箱和注册时间等字段，支持对用户角色的编辑修改和过期账号的删除操作。系统同时记录关键操作的审计日志，包括登录、登出、配置修改和告警确认等操作，便于安全审计和问题追溯。

5.3.2 数据接入与管理模块

数据接入与管理模块支持文件上传和 Prometheus API 接入两种数据输入方式，并提供统一的数据集管理功能。

文件上传方式支持 CSV 和 JSON 两种格式。CSV 格式支持宽表（列为节点）和长表（包含 timestamp、node、value 列）两种结构，系统自动检测格式并转换为统一的 DataFrame 结构。JSON 格式采用嵌套字典结构，支持多指标数据的组织。文件上传接口通过 Flask 的文件处理机制实现，限制最大上传大小为 50MB，仅允许指定扩展名的文件类型。

Prometheus API 接入通过 HTTP 请求调用 Prometheus 的 query_range 接口，支持标准的 PromQL 查询语法。系统自动解析返回的 JSON 数据，提取节点标识 (instance 标签) 和时序数值，转换为统一的数据格式。Prometheus 数据接入的核心实现如下：

```

1 def load_from_prometheus(url, query, start, end, step='60s'):
2     api_url = f"{url.rstrip('/')}/api/v1/query_range"
3     params = {'query': query, 'start': start,
4               'end': end, 'step': step}
5     response = requests.get(api_url, params=params, timeout=30)
6     data = response.json()
7     results = {}
8     for result in data['data']['result']:
9         labels = result['metric']

```

```

node_name = labels.get('instance', 'unknown')
11 timestamps = [float(v[0]) for v in result['values']]
values = [float(v[1]) for v in result['values']]
13 results[node_name] = values
return pd.DataFrame(results,
15 index=pd.to_datetime(timestamps, unit='s'))

```

Listing 1 Prometheus 数据接入核心代码

两种数据接入方式的输出统一为字典格式 {metric_name: DataFrame}，其中 DataFrame 的列为节点名称，行索引为时间戳，确保后续预处理和检测模块无需关注数据来源的差异。

在数据管理页面中，用户可以查看已上传数据集的列表信息，包括数据集名称、数据规模（节点数和时间步数）、上传时间等。模块内置了时序数据预览功能，用户点击某个数据集后，页面右侧自动渲染该数据集的时序曲线图，以折线图形式展示各节点的指标变化趋势，支持通过下拉框切换不同指标的预览视图。

5.3.3 预处理服务模块

预处理服务模块对原始监控数据进行系统性清洗与标准化处理，实现为完整的预处理流水线。

数据对齐采用基于时间戳的重采样方法，通过 Pandas 的 resample 接口将不规则采样的数据转换为固定间隔的时序数据，聚合策略采用均值。缺失值处理采用分段策略：对短时缺失（连续缺失不超过 5 个采样点）采用线性插值方法进行填充，对长时间连续缺失的节点数据直接剔除以避免引入较大误差。数据归一化支持 Z-score 标准化、Min-Max 归一化和基于中位数的鲁棒归一化三种方法，用户可根据数据特点在配置页面中选择合适的归一化策略。

预处理流水线将上述步骤串联执行，核心实现如下：

```

1 def preprocess_pipeline(data_dict, interval='60s',
    normalize_method='zscore'):
3     processed = {}
    for metric_name, df in data_dict.items():
5         # Step 1: 时间戳对齐与重采样
        df_aligned = align_timestamps(df, interval)
7         # Step 2: 缺失值检测与分段填充
        df_filled, dropped = interpolate_missing(
9             df_aligned, max_gap=5)
        # Step 3: 数据归一化
11        df_normalized, stats = normalize(
            df_filled, normalize_method)
13        processed[metric_name] = df_normalized
    return processed

```

Listing 2 预处理流水线核心代码

在预处理界面中，用户可以选择待处理的数据集并配置预处理参数，包括重采样间隔、缺失值填充方法和归一化方式等。系统在预处理过程中记录详细的处理日志，用户可在预处理日志页面中查看每个步骤的执行时间和处理结果，如时间戳对齐后的数据点数、被剔除的缺失节点列表和归一化统计参数等信息。

5.3.4 检测引擎模块

检测引擎模块采用模块化设计，将每种算法封装为独立的检测器类，通过统一的 `detect()` 接口进行调用。检测引擎根据用户配置的场景类型自动选择对应的检测流程。

对于单指标多节点场景，检测引擎依次调用欧氏距离检测器、自编码器嵌入检测器和 DBSCAN 聚类检测器，获取三种方法各自的异常分数后，通过 I-SPOT 算法为每种方法计算自适应阈值，最后执行保守集成决策。欧氏距离检测器的核心计算采用 SciPy 的 `pdist` 函数实现向量化的成对距离计算，避免了显式的双重循环，将时间复杂度从 $O(N^2 \cdot T)$ 优化至接近 $O(N^2)$ 的矩阵运算效率：

```

def compute_anomaly_scores(self, data):
2   T, N = data.shape
   n_windows = T - self.window_size + 1
4   scores = np.zeros((n_windows, N))
   for t in range(n_windows):
6       window = data[t:t + self.window_size, :]
       # 向量化计算节点间欧氏距离矩阵
8       dist_matrix = squareform(
           pdist(window.T, metric='euclidean'))
10      np.fill_diagonal(dist_matrix, 0)
       avg_distances = dist_matrix.sum(axis=1) / (N - 1)
12      # Z-score 归一化
       mu, sigma = avg_distances.mean(), avg_distances.std()
14      if sigma > 1e-10:
           scores[t] = (avg_distances - mu) / sigma
16      return scores

```

Listing 3 欧氏距离检测器核心代码

对于多指标多节点场景，检测引擎并行运行数值检测模块和形状检测模块。数值检测模块对各指标分别计算欧氏距离异常分数后进行加权融合；形状检测模块采用改进 SAX 算法提取各指标的形状特征并计算异常分数。最终通过可配置的权重参数融合两种视角的检测结果。

在检测配置页面中，用户可以选择检测所使用的场景类型（单指标或多指标）、指定数据来源、配置各算法的关键参数（如滑动窗口大小、I-SPOT 初始化长度、集成决策投票比例等），并设置异常判定阈值。系统提交检测任务后自动执行完整的检测流程，用户可在检测记录页面中查看所有历史检测任务的执行状态和结果摘要，包括任务类型、关联集群、检测时间、发现的异常节点数量等信息。

5.3.5 常态化监控模块

常态化监控模块是系统服务于生产环境的核心功能，通过定时任务调度机制实现对分布式集群的持续性自动巡检，将异常检测从被动的人工触发模式转变为主动的自动化监控模式。

模块的核心实现基于 APScheduler 定时任务框架，结合集群配置表中的巡检参数，为每个已启用的被监控集群创建独立的定时任务。定时任务按照配置的

巡检间隔（默认每 5 分钟一次）自动执行以下流程：首先，通过 Prometheus API 拉取该集群最近一个检测窗口内的监控数据；然后，调用预处理流水线对原始数据进行清洗和标准化；接着，调用检测引擎执行配置的异常检测算法；最后，将检测结果写入数据库，并触发告警规则匹配流程。常态化监控的调度核心实现如下：

```

1 from apscheduler.schedulers.background import BackgroundScheduler
2
3 scheduler = BackgroundScheduler()
4
5 def init_monitoring_jobs():
6     """根据数据库配置初始化所有集群的巡检任务"""
7     clusters = ClusterConfig.query.filter_by(
8         is_active=True).all()
9     for cluster in clusters:
10         scheduler.add_job(
11             func=run_detection_cycle,
12             trigger='interval',
13             minutes=cluster.check_interval,
14             id=f'monitor_{cluster.cluster_id}',
15             args=[cluster.cluster_id],
16             replace_existing=True
17         )
18     scheduler.start()
19
20 def run_detection_cycle(cluster_id):
21     """执行单次巡检周期的完整流程"""
22     cluster = ClusterConfig.query.get(cluster_id)
23     end_time = datetime.now()
24     start_time = end_time - timedelta(
25         minutes=cluster.check_interval * 10)
26     # Step 1: 拉取Prometheus数据
27     raw_data = load_from_prometheus(
28         cluster.prometheus_url,
29         cluster.promql_query,
30         start_time.timestamp(),
31         end_time.timestamp()
32     )
33     # Step 2: 预处理
34     processed = preprocess_pipeline(
35         {'metric': raw_data})
36     # Step 3: 异常检测
37     results = detection_engine.detect(
38         processed, cluster.scenario_type)
39     # Step 4: 保存结果并触发告警检查
40     save_results(cluster_id, results)
41     check_alert_rules(cluster_id, results)

```

Listing 4 常态化监控调度核心代码

常态化监控模块在系统启动时自动读取数据库中所有已启用集群的配置信息，并为每个集群注册对应的定时巡检任务。当管理员在集群配置页面中新增、修改或停用某个集群的监控配置时，系统通过动态更新 APScheduler 的任务列表实现巡检策略的实时生效，无需重启服务。

在集群监控管理页面中，管理员可以查看所有被监控集群的列表信息，包括集群名称、类型、Prometheus 地址、巡检间隔、最近一次巡检时间和监控状态

等。管理员可以新增集群配置，编辑现有集群的监控参数（如修改 PromQL 查询表达式、调整巡检间隔），或临时启停某个集群的自动巡检。页面同时展示各集群的最近巡检结果摘要，包括检测到的异常节点数量和最高异常分数，帮助管理员快速掌握各集群的整体健康状况。

5.3.6 告警管理模块

告警管理模块是系统将异常检测结果转化为运维行动的关键环节，实现了从告警规则配置、告警触发判定、多渠道通知分发到告警生命周期管理的完整闭环。

5.3.6.1 告警规则配置

系统支持按集群维度配置差异化的告警规则。每条告警规则包含告警级别、触发阈值和通知方式三个核心要素。告警级别分为三级：信息级（level=1）、警告级（level=2）和严重级（level=3），对应不同程度的异常严重性。每个级别可以独立设置异常分数阈值，例如设定异常分数超过 2.0 触发信息级告警、超过 3.0 触发警告级告警、超过 4.5 触发严重级告警。

在告警规则配置页面中，用户可以为每个被监控集群创建多条告警规则。每条规则的配置项包括：（1）告警级别，通过下拉框选择信息、警告或严重三个级别；（2）异常分数阈值，当某节点的异常评分超过该值时触发对应级别的告警；（3）通知方式，支持邮件通知、企业微信 Webhook 和仅系统内记录三种方式；（4）通知目标，根据通知方式填写邮件地址或 Webhook URL。系统同时支持告警规则的启用和停用操作，停用后的规则不参与告警判定但保留配置信息。

5.3.6.2 告警触发与通知

告警触发流程在每次检测任务完成后自动执行。系统遍历检测结果中每个节点的异常评分，与该集群关联的所有已启用告警规则进行逐一匹配。当某节点的异常分数超过某条规则的阈值时，系统创建一条告警记录并根据规则配置的通知方式发送告警通知。

对于邮件通知方式，系统通过 SMTP 协议构造包含告警详情的 HTML 邮件，内容涵盖告警级别、集群名称、异常节点、异常分数、检测时间和建议处置措施等信息，发送至规则配置的邮件地址列表。对于企业微信 Webhook 通知方式，系统将告警信息格式化为 Markdown 格式的消息体，通过 HTTP POST 请求推送至配置的 Webhook 地址。通知发送服务采用异步执行机制，通过 Python 的 threading 模块将通知任务提交至独立线程执行，避免通知延迟阻塞主检测流程。告警通知的核心实现如下：

```
1 def check_alert_rules(cluster_id, results):  
2     """检查告警规则并分发通知"""  
3     rules = AlertRule.query.filter_by(  
4         cluster_id=cluster_id, is_enabled=True).all()  
5     for node, score in results['scores'].items():
```



```

    for rule in rules:
        if score >= rule.score_threshold:
            # 创建告警记录
            log = AlertLog(
                task_id=results['task_id'],
                alert_level=rule.alert_level,
                node_name=node,
                anomaly_score=score,
                alert_message=generate_message(
                    rule.alert_level, node, score),
                alert_time=datetime.now()
            )
            db.session.add(log)
            # 异步发送通知
            threading.Thread(
                target=send_notification,
                args=(rule.notify_method,
                    rule.notify_target, log)
            ).start()
    db.session.commit()

```

Listing 5 告警通知分发核心代码

为避免告警风暴，系统实现了告警抑制机制：对于同一集群的同一节点，在可配置的抑制时间窗口内（默认 30 分钟），相同级别的告警仅触发一次通知。该机制通过在告警触发前查询最近的告警记录实现，有效减少了重复告警对运维人员的干扰。

5.3.6.3 告警日志与生命周期管理

告警日志页面提供完整的告警历史记录查询功能。页面顶部设置了多维度的筛选条件，包括告警级别（信息/警告/严重）、告警状态（未确认/已确认）、时间范围和关键词搜索等，支持运维人员快速定位特定的告警事件。告警列表以表格形式展示，每条记录包含告警时间、告警级别（以不同颜色标签区分）、集群名称、异常节点、异常分数、告警信息和通知状态等字段。

系统为每条告警记录设计了完整的生命周期管理流程。告警记录创建后初始状态为“未确认”，运维人员查看告警详情并采取相应处置措施后，可以点击“确认”按钮将告警状态更新为“已确认”，同时可以填写处置备注信息。这一机制确保每条告警都能被追踪到具体的响应人和处理结果，满足企业运维流程的合规要求。

告警管理页面同时提供告警统计分析功能。系统按告警级别、集群、时间维度对告警记录进行聚合统计，以柱状图和饼图等形式展示告警趋势和分布情况，帮助运维管理者从宏观层面评估集群的稳定性和告警规则的合理性。

5.3.7 可视化与数据看板模块

可视化与数据看板模块通过交互式图表和数据面板，为运维人员提供多层次、多维度的集群健康状态视图。模块基于 ECharts 图表库实现，支持丰富的交互操作和自适应的数据展示。

5.3.7.1 实时监控大屏

实时监控大屏是运维人员日常监控的主入口页面，集中展示所有被监控集群的实时健康状态。页面顶部的状态概览区域以卡片形式显示关键统计指标，包括当前监控的集群数量、最近一小时内检测到的异常节点总数、未处理的告警数量和系统运行时长等信息。页面中部的集群健康状态面板以彩色状态标签展示各集群的最近巡检结果：绿色表示全部节点正常，黄色表示存在信息级或警告级异常，红色表示存在严重级异常。运维人员可以点击某个集群卡片快速跳转至该集群的详细检测页面。

5.3.7.2 多节点曲线对比分析

多节点曲线对比图是系统最核心的可视化视图，用于直观展示异常节点与正常节点群体行为模式之间的偏离情况。图表采用 ECharts 折线图组件绘制所有节点的时序曲线，异常节点以红色加粗线条高亮显示，正常节点以低透明度的蓝色线条作为背景参照。图表支持鼠标滚轮缩放和拖拽平移操作，便于运维人员查看不同时间段的详细信息。图表下方的时间轴滑块支持快速选择关注的时间范围。

5.3.7.3 异常分数排名与热力图

异常分数排名图以水平柱状图的形式展示各节点的异常评分，按分数从高到低排列。异常分数较高的节点以红色标注，正常范围内的节点以蓝色标注，阈值线以虚线形式标记在图表中，帮助运维人员直观判断各节点的异常程度。

异常检测时间线热力图以节点为纵轴、时间步为横轴，通过颜色深浅编码异常分数的大小。浅色表示正常状态，深红色表示高异常分数。该视图能够全局展示异常在时间和节点两个维度上的分布规律，帮助发现异常事件的时空关联性，识别是单节点持续性异常还是多节点短时爆发性异常等不同异常模式。

5.3.7.4 历史检测结果回溯

历史检测结果页面支持运维人员按时间范围筛选并查看过去的检测记录。页面上方提供日期选择器和集群筛选框，下方以时间轴的形式展示历史检测结果。每条记录显示检测时间、异常节点数量和最高异常分数等摘要信息，点击可展开查看完整的检测详情，包括所有节点的异常评分、异常判定结果和对应的可视化图表。历史数据支持导出为 CSV 格式文件，便于进一步的离线分析和报告编写。

5.4 系统应用效果展示

系统已在联通大数据生产底座平台上完成部署并投入运行，对接了多个 YARN 集群和 GPU 集群的 Prometheus 监控数据，实现了对关键业务指标的常态化自动巡检。以下分别从系统性能、常态化监控运行效果和告警系统运行效果三个维度展示系统的实际应用表现。

5.4.1 系统性能评估

在性能方面，系统能够支持对包含数十个节点的集群进行秒级响应的异常检测。表5-8展示了不同规模数据下各检测算法的执行耗时。欧氏距离检测器得益于 SciPy 的向量化实现，即使在 50 节点、1000 时间步的场景下也能在 3 秒内完成检测。自编码器检测器的训练阶段耗时较长，但推理阶段的耗时与欧氏距离检测器相当。通过内存缓存策略，重复检测同一数据集时的响应时间可缩短至首次检测的 20% 以下。

表 5-8 系统检测性能（单位：秒）

数据规模	欧氏距离	自编码器	DBSCAN	集成决策
15 节点 ×500 步	0.8	12.5	1.2	14.8
30 节点 ×500 步	1.5	15.3	2.1	19.4
50 节点 ×1000 步	3.2	28.7	4.5	37.1

在常态化监控场景下，系统需要在每个巡检周期内完成数据拉取、预处理和检测的完整流程。对于典型的 YARN 集群（15 至 30 个节点，巡检间隔 5 分钟，检测窗口 50 分钟即 500 个时间步），单次巡检的端到端耗时约为 20 秒，远低于 5 分钟的巡检间隔，确保系统能够稳定支持多个集群的并行巡检。表5-9展示了端到端巡检的各阶段耗时分布。

表 5-9 常态化巡检端到端耗时分布（30 节点 ×500 步）

处理阶段	平均耗时（秒）	占比
Prometheus 数据拉取	1.2	6.2%
数据预处理	0.8	4.1%
异常检测（集成决策）	16.5	85.1%
结果存储与告警检查	0.9	4.6%
总计	19.4	100%

5.4.2 常态化监控运行效果

系统在联通大数据平台上对多个 YARN 集群的 RPC Router 连接数、ResourceManager 活跃节点数、NodeManager 内存使用率等关键指标进行了为期两周的常态化监控验证。验证期间，系统累计执行超过 4000 次自动巡检任务，成功识别出多起节点级异常事件。

在实际运行中，系统检测到的典型异常事件包括：因网络隔离导致的单节点连接数突降、因资源竞争引发的个别节点内存使用率异常飙升、以及因配置变更造成的部分节点指标偏移等。相比传统的固定阈值告警方式，基于曲线相似性的检测方法能够有效区分集群整体负载变化（如业务高峰期所有节点连接数同步

上升)和个别节点异常(如单节点在群体下降趋势中逆势上升),显著降低了误报率。

系统的核心优势在于:第一,通过多节点曲线对比的方式定义异常,不依赖对“正常范围”的先验定义,因此对负载敏感型指标具有更强的鲁棒性。第二,保守集成决策策略通过要求多种方法同时确认异常来降低误报率,适合对告警精度要求较高的生产环境。第三,I-SPOT 自适应阈值算法自动跟踪数据分布的变化,无需人工调整阈值参数,减轻了运维人员的配置负担。第四,常态化监控模式实现了从被动响应到主动发现的转变,将异常发现的平均时间从依赖人工巡检的小时级缩短至 5 分钟级,显著提升了运维效率。

5.4.3 告警系统运行效果

在两周的验证运行期间,告警系统累计触发 73 条告警记录,其中信息级告警 41 条、警告级告警 24 条、严重级告警 8 条。严重级告警均对应真实的节点故障或异常事件,人工核实确认率达到 100%;警告级告警的人工确认率为 87.5%,少数未确认的记录经分析属于短暂性指标波动;信息级告警主要用于记录轻微偏差,为运维人员提供趋势参考。

告警通知的发送延迟方面,邮件通知的平均发送延迟为 2.3 秒,企业微信 Webhook 通知的平均发送延迟为 0.8 秒,均能满足生产环境对告警时效性的要求。告警抑制机制在验证期间有效过滤了约 120 条重复告警通知,将实际推送的通知数量控制在合理范围内,避免了告警疲劳问题。

通过将异常检测算法与常态化监控和多级告警机制相结合,系统为运维团队提供了一套完整的“监测、发现、告警、处置”闭环解决方案,有效提升了分布式集群的运维自动化水平和故障响应速度。

5.5 本章小结

本章详细介绍了基于曲线相似性的异常检测系统的设计与实现过程,涵盖系统功能模块设计、分层架构设计、数据库设计、开发环境与技术选型以及各核心模块的具体实现。

在系统设计方面,系统划分为用户与权限管理、数据接入与管理、数据预处理、检测引擎、常态化监控、告警管理以及可视化与数据看板七个功能模块,采用用户层、应用层、算法层和数据层的四层架构实现前后端分离。数据库设计了用户表、集群配置表、检测任务表、检测结果表、告警规则表和告警记录表六张核心数据表,完整覆盖了系统的业务数据存储需求。

在系统实现方面,数据接入模块支持文件上传和 Prometheus API 两种数据源,预处理模块实现了包含时间戳对齐、缺失值处理和归一化的完整流水线,检测引擎以模块化设计封装了本文提出的全部检测算法。常态化监控模块通过 APScheduler 定时任务框架实现了对多个集群的 7×24 小时自动巡检,将异常检测从手动触发模式升级为自动化持续监控模式。告警管理模块构建了三级告警

机制，支持邮件和企业微信等多渠道通知，并实现了告警抑制、告警确认和告警统计等生命周期管理功能。可视化模块提供了实时监控大屏、多节点曲线对比、异常分数排名、时间线热力图和历史结果回溯等多种视图。

在应用验证方面，系统已在联通大数据生产底座平台上完成部署并通过两周的常态化运行验证。系统在典型集群规模下的单次巡检端到端耗时约为 20 秒，满足 5 分钟巡检间隔的性能要求；告警系统的严重级告警人工确认率达到 100%，告警通知延迟控制在秒级，验证了系统在实际生产环境中的可行性和实用价值。

第6章 总结与展望

6.1 本文工作总结

分布式集群是支撑现代云计算与大数据处理的核心基础设施，其稳定性直接关系到上层业务的连续性。针对集群异常检测中现有方法未能有效利用多节点行为相似性这一重要先验知识的问题，本文提出了基于多元时序曲线相似性的分布式集群异常检测方法。本文的主要工作和贡献总结如下。

第一，本文提出了一种基于多节点曲线相似性的异常检测方法论。该方法突破了传统异常检测聚焦于时间与指标二维分析的框架，创新性地将集群节点这一空间维度系统性地引入模型，构建了时间、指标与节点的三维分析范式。通过利用分布式集群负载均衡机制所保证的节点行为相似性，将隐性的运维经验转化为显式的可计算先验知识，有效规避了因定义绝对正常模式而产生的误报问题。

第二，针对单指标多节点场景，本文基于联通大数据生产环境构建了高质量的时序数据集，并分别采用欧氏距离、自编码器嵌入和 DBSCAN 聚类三种方法度量节点曲线间的相似性。针对单一方法阈值敏感的问题，设计了基于逻辑与策略的集成学习框架，并引入改进的包容性 SPOT 算法实现阈值的动态自适应调整。实验结果表明，所提方法在 F1 分数上显著优于现有的多元时序异常检测算法。

第三，针对多指标多节点场景，本文提出了数值与形状双视角融合的异常检测框架。在数值视角方面，采用基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面，提出了改进的 SAX 符号化表示算法，通过保留原始时间分辨率和采用严格距离计算规则来提升对形状变化的敏感性。在 GPU 集群故障数据集上的实验表明，融合方法的故障检测准确率达到 83.8%，显著优于单一视角方法和传统基线方法。

第四，本文设计并实现了基于曲线相似性的异常检测系统，实现了从数据采集、预处理、异常检测到结果展示的完整流程。系统已在联通大数据生产底座平台上进行部署验证，成功识别出多起节点级异常事件，验证了方法在实际生产环境中的可行性和有效性。

6.2 未来研究展望

虽然本文在基于多节点曲线相似性的异常检测方面取得了一定进展，但仍有若干值得深入研究的方向。

第一，更复杂的节点关系建模。本文假设同构集群内各节点地位平等，但实际系统中节点可能存在主从关系、依赖关系等复杂结构。未来可以引入图神经网络等方法显式建模节点间的拓扑关系，提升对复杂系统结构的刻画能力。

第二，异构集群的异常检测。本文主要关注同构集群中节点行为的相似性，但在异构集群中，不同类型节点的正常行为模式可能存在差异。未来可以研究如何在异构环境中识别可比较的节点组，并设计适应性更强的检测方法。

第三，在线增量学习能力。当前方法主要采用批量处理模式，对于持续运行的系统，需要能够在线更新模型以适应系统行为的长期演变。未来可以研究增量学习方法，使模型能够在保持对历史模式记忆的同时适应新的正常行为模式。

第四，异常根因分析。当前方法主要关注异常检测和故障节点定位，对于异常的具体原因缺乏深入分析。未来可以结合日志分析、配置变更追踪等信息，构建从异常检测到根因定位的完整链路，进一步提升系统的智能运维能力。

第五，跨集群知识迁移。不同集群的监控数据虽然在绝对值上可能存在差异，但其中蕴含的异常模式可能具有相似性。未来可以研究如何在多个集群之间进行知识迁移和经验共享，提升检测方法在新场景中的泛化能力。

参考文献

- [1] Grubbs F E. Procedures for detecting outlying observations in samples [J]. *Technometrics*, 1969, 11(1): 1-21.
- [2] Dixon W J. Analysis of extreme values [J]. *The Annals of Mathematical Statistics*, 1950, 21(4): 488-506.
- [3] Mahalanobis P C. On the generalised distance in statistics [J]. *Proceedings of the National Institute of Science of India*, 1936, 2: 49-55.
- [4] Box G E, Jenkins G M. *Time series analysis: Forecasting and control* [M]. Holden-Day, 1970.
- [5] Cleveland R B, Cleveland W S, McRae J E, et al. Stl: A seasonal-trend decomposition procedure based on loess [J]. *Journal of Official Statistics*, 1990, 6(1): 3-73.
- [6] Ramaswamy S, Rastogi R, Shim K. Efficient algorithms for mining outliers from large data sets [C]//*Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. 2000: 427-438.
- [7] Breunig M M, Kriegel H P, Ng R T, et al. Lof: identifying density-based local outliers [J]. *ACM SIGMOD Record*, 2000, 29(2): 93-104.
- [8] Liu F T, Ting K M, Zhou Z H. Isolation forest [C]//*Proceedings of the 8th IEEE International Conference on Data Mining*. 2008: 413-422.
- [9] Choi K, Yi J, Park C, et al. Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines [J]. *IEEE Access*, 2021, 9: 120043-120065.
- [10] Pang G, Shen C, Cao L, et al. Deep learning for anomaly detection: A review [J]. *ACM Computing Surveys*, 2021, 54(2): 1-38.
- [11] Su Y, Zhao Y, Niu C, et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network [C]//*Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019: 2828-2837.
- [12] Audibert J, Michiardi P, Guyard F, et al. Usad: Unsupervised anomaly detection on multivariate time series [C]//*Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2020: 3395-3404.
- [13] Hochreiter S, Schmidhuber J. Long short-term memory [J]. *Neural computation*, 1997, 9(8): 1735-1780.
- [14] Tuli S, Casale G, Jennings N R. Tranad: Deep transformer networks for anomaly detection in multivariate time series data [C]//*Proceedings of the VLDB Endowment: volume 15*. 2022: 1201-1214.
- [15] Xu J, Wu H, Wang J, et al. Anomaly transformer: Time series anomaly detection with association discrepancy [J]. *arXiv preprint arXiv:2110.02642*, 2022.
- [16] Deng A, Hooi B. Graph neural network-based anomaly detection in multivariate time series [C]//*Proceedings of the AAAI Conference on Artificial Intelligence: volume 35*. 2021: 4027-4035.
- [17] Chen W, Liu Y, Zhang M. Carots: Causal-aware contrastive learning for robust multivariate time series anomaly detection [C]//*Proceedings of the 42nd International Conference on Machine Learning*. 2025.
- [18] Ren H, Xu B, Wang Y, et al. Time-series anomaly detection service at Microsoft [C]//

- Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2019: 3009-3017.
- [19] Darban Z Z, Webb G I, Pan S, et al. Carla: Self-supervised contrastive representation learning for time series anomaly detection [J]. *Pattern Recognition*, 2025, 157: 110874.
 - [20] Paparrizos J, Kang Y, Boniol P, et al. Tsb-uad: An end-to-end benchmark suite for univariate time-series anomaly detection [J]. *Proceedings of the VLDB Endowment*, 2022, 15(8): 1697-1711.
 - [21] Wu Q, Jiang T, Zhang Y. Tab: Unified benchmarking of time series anomaly detection methods [J]. *arXiv preprint arXiv:2506.18046*, 2024.
 - [22] Fan X, Li F, Qi L. Distributed discord discovery: Spark based anomaly detection in time series [C]//*Proceedings of the IEEE International Conference on High Performance Computing and Communications*. 2015: 154-159.
 - [23] Carbone P, Katsifodimos A, Ewen S, et al. Apache Flink: Stream and batch processing in a single engine [J]. *IEEE Data Engineering Bulletin*, 2015, 36(4): 28-38.
 - [24] Ester M, Kriegel H P, Sander J, et al. A density-based algorithm for discovering clusters in large spatial databases with noise [J]. *KDD*, 1996, 96(34): 226-231.
 - [25] Kingma D P, Welling M. Auto-encoding variational bayes [J]. *arXiv preprint arXiv:1312.6114*, 2014.
 - [26] Zhang H, Wang Y, Huang T. Tfmae: Temporal-frequency masked autoencoder for time series anomaly detection [C]//*Proceedings of the 40th IEEE International Conference on Data Engineering*. 2024: 1-12.
 - [27] Zhao H, Wang Y, Duan J, et al. Multivariate time-series anomaly detection via graph attention network [C]//*Proceedings of the 2020 IEEE International Conference on Data Mining*. 2020: 841-850.
 - [28] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks [C]//*International Conference on Learning Representations*. 2018.
 - [29] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks [J]. *arXiv preprint arXiv:1609.02907*, 2017.
 - [30] Wu Z, Pan S, Chen F, et al. A comprehensive survey on graph neural networks [J]. *IEEE Transactions on Neural Networks and Learning Systems*, 2021, 32(1): 4-24.
 - [31] Zhang J, Chen H, Li Y. Graph spatiotemporal process for multivariate time series anomaly detection with missing values [J]. *arXiv preprint arXiv:2401.05800*, 2024.
 - [32] Liu Y, Wang H, Zhang Q. Mgcl: Multiorder graph neural network with cross-learning for multivariate time-series anomaly detection [J]. *IEEE Transactions on Instrumentation and Measurement*, 2025.
 - [33] Chen X, Huang W, Wang Z. Entropy causal graphs for multivariate time series anomaly detection [J]. *ACM Transactions on Knowledge Discovery from Data*, 2025.
 - [34] Li D, Chen D, Shi L, et al. Mad-gan: Multivariate anomaly detection for time series data with generative adversarial networks [C]//*International Conference on Artificial Neural Networks*. 2019: 703-716.
 - [35] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets [C]//*Advances in Neural Information Processing Systems: volume 27*. 2014.
 - [36] Boniol P, Palpanas T. Distributed detection of sequential anomalies in univariate time series [J]. *The VLDB Journal*, 2022, 31: 233-256.

附录一 附录中的公式

对公式的引用如，公式(附 1-1)

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad (\text{附 1-1})$$

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad (\text{附 1-2})$$

mathtext: $A, F, L, 2, 3, 5, \sigma$, mathnormal: $A, F, L, 2, 3, 5, \sigma$, mathrm: $A, F, L, 2, 3, 5, \sigma$.

mathbf: $\mathbf{A}, \mathbf{F}, \mathbf{L}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \boldsymbol{\sigma}$, mathit: $A, F, L, 2, 3, 5, \sigma$, mathsf: $A, F, L, 2, 3, 5, \sigma$.

mathtt: $A, F, L, 2, 3, 5, \sigma$, mathfrak: $\mathfrak{A}, \mathfrak{F}, \mathfrak{L}, 2, 3, 5, \sigma$, mathbb: $\mathbb{A}, \mathbb{F}, \mathbb{L}, 2, 3, 5, \sigma$.

mathcal: $\mathcal{A}, \mathcal{F}, \mathcal{L}, 2, 3, 5, \sigma$, mathscr: $\mathscr{A}, \mathscr{F}, \mathscr{L}, 2, 3, 5, \sigma$, boldsymbol: $\mathbf{A}, \mathbf{F}, \mathbf{L}, 2, 3, 5, \boldsymbol{\sigma}$.

vector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitvector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

matrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitmatrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

tensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unittensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

附录二 附录中的图表

附表测试

附表 2-1 这是一个样表
App Table 2-1 This is a sample table

行号	跨多列的标题							
Row 1	1	2	3	4	5	6	7	8

附表 2-2 这是一个样表
App Table 2-2 This is a sample table

行号	跨多列的标题							
Row 1	1	2	3	4	5	6	7	8

附图测试



附图 2-1 这是一个样图
App Figure 2-1 This is a sample figure

注: 对图片的注释

致 谢

2026 年 6 月

作者简历及攻读学位期间发表的学术论文与其他相关学术成果

作者简历：

已发表（或正式接受）的学术论文：

参加的研究项目及获奖情况：

